

SAPIENZA
UNIVERSITÀ DI ROMA

# Lightweight Scene Understanding and Synthetic Data-Driven Damage Inspection for Martian Rovers


Faculty of Information Engineering, Informatics, and Statistics
Master's Degree in Computer Science

**Daniele Venturini**
ID number 1991255

Advisor
Prof. Marco Raoul Marini


Academic Year 2025/2026

Thesis defended on 23 October 2026
in front of a Board of Examiners composed by:

Prof. Marco Raoul Marini (chairman, Thesis Advisor)

---

**Lightweight Scene Understanding and Synthetic Data-Driven Damage Inspection for Martian Rovers**

Master's Degree Thesis. Sapienza University of Rome

© 2026 Daniele Venturini. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Version: 1.048596

Author's email: venturini.1991255@studenti.uniroma1.it

*Dedicato a*
*me*

## Abstract

Questa tesi parla di me.

# Contents

**1 Introduction** **1**
1.1 Problem Statement . . . 1
1.2 State of the Art . . . 1
1.2.1 Datasets . . . 1
1.3 Proposed Research Framework . . . 2
1.4 Main Contributions . . . 2
1.5 Thesis Organization . . . 2

**2 Lightweight Scene Understanding** **3**
2.1 Task Definition . . . 3
2.2 Dataset . . . 3
2.3 Pre-processing . . . 5
2.4 Segmentation Architectures . . . 5
2.4.1 U-Net Variants . . . 6
2.4.2 DeepLab Variants . . . 8
2.4.3 SegFormer-B0 . . . 10
2.4.4 LCNet Variants . . . 11
2.4.5 Architectural Comparison . . . 12
2.5 Training . . . 13
2.6 Evaluation Metrics . . . 14
2.7 Optimization and Quantization . . . 15

**3 Synthetic Dataset Generation** **17**
3.1 Design Requirement . . . 17
3.2 Assets preparation . . . 17
3.3 Synthetic Damage Generation . . . 19
3.4 Domain Randomization . . . 19
3.5 Quality Assurance . . . 21
3.6 Final Dataset Composition . . . 22

**4 Synthetic Data-Driven Damage Inspection** **25**
4.1 Task Definition . . . 25
4.2 Pre-processing . . . 25
4.3 Anomaly Detection Models . . . 26
4.3.1 PatchCore . . . 27
4.3.2 EfficientAD . . . 28

4.3.3 SuperSimpleNet . . . 29
4.3.4 TinyGLASS . . . 30
4.4 Evaluation Metrics . . . 31

**5 Evaluation of Lightweight Scene Understanding 33**
5.1 Experimental Setup . . . 33
5.2 Segmentation Accuracy . . . 33
5.3 Quantization Results . . . 35
5.4 Ablation Studies . . . 37
5.4.1 Loss Function and Class Weighting . . . 38
5.4.2 Effect of Pretraining . . . 39
5.4.3 Effect of Data Augmentation . . . 39
5.5 Cross-Mission Generalization on MarsSeg . . . 40

**6 Evaluation of Synthetic Damage Inspection 43**
6.1 Experimental Setup . . . 43
6.2 Model Comparison . . . 44
6.3 Ablation Studies . . . 48
6.3.1 Memory Bank and Smoothing in PatchCore . . . 48
6.3.2 Calibration and Scoring in EfficientAD-S . . . 49
6.3.3 Training and Scoring in SuperSimpleNet . . . 50
6.3.4 Task Adaptation and Spatial Resolution in TinyGLASS . . . 52

**7 Discussion and Limitations 55**
7.1 Main Findings . . . 55
7.2 Sim-to-Real Gap . . . 55

**8 Conclusions and Future Work 57**

**Bibliography 59**

# Chapter 1

# Introduction

## 1.1 Problem Statement

## 1.2 State of the Art

Wheel perforations are a documented form of damage experienced by NASA's Curiosity rover. Official images acquired by the Mars Hand Lens Imager (MAHLI) show holes and tears in the aluminum wheel skin, which have been monitored through periodic wheel inspections [26]. In the ground-based procedure described by Rankin et al., the Wheel Wear team examines downlinked images, compares them with previous observations, and manually measures damage features to track their progression [27]. This operational context motivates investigating automated visual inspection as a means of supporting damage detection and localization. However, no available dataset was suitable for this task. For example, the public MSL Curiosity image-classification dataset includes a generic wheel category, but provides image-level content labels rather than annotations of individual perforations or pixel-level damage masks [28]. Among the public resources examined for this work, no ready-to-use benchmark was identified that combined hole specific annotations with the experimental task required by the proposed damage inspection approach, meaning a dedicated dataset was needed. Within the time available for this work, synthetic generation was selected instead of manually curating and densely annotating a real-image corpus. This choice also enabled controlled variation of the damage and imaging conditions, with annotations derived directly from the generated scenes.

### 1.2.1 Datasets

AI4Mars was considered as an alternative because it is one of the most established large-scale resources for Martian terrain segmentation. Its authors report approximately 35,000 rover images and 326,000 full-image annotations, with multiple crowdsourced annotations collected for each image and a smaller set of approximately 1,500 expert-produced validation labels [5]. Its semantic scope, however, is limited to four terrain categories: *Soil*, *Bedrock*, *Sand*, and *Big Rock*. Pixels overlapping the rover are removed from the terrain annotations through auxiliary masks rather than being represented by a trainable *Rover* category. AI4Mars therefore provides neither

the range of environmental categories available in the selected S$^5$Mars release nor the rover-level semantic information relevant to the broader aims of this thesis.

Annotation quality provided a further reason not to adopt AI4Mars as the primary benchmark. Following an expert inspection, the Mars-Bench authors reported annotation errors in AI4Mars and excluded it from their benchmark, arguing that crowdsourced annotations posed difficulties in a specialized planetary-science setting [3]. This finding is treated here as a dataset-selection concern rather than evidence that AI4Mars is unusable in general. The original AI4Mars pipeline included agreement-based filtering, manual quality control, multiple annotations per image, and an expert-labelled validation subset [5]. The decision made in this work therefore rests on the combination of its narrower taxonomy, the absence of a semantic *Rover* category, and the specific annotation concerns raised by the Mars-Bench expert review.

## 1.3 Proposed Research Framework

## 1.4 Main Contributions

## 1.5 Thesis Organization

# Chapter 2

# Lightweight Scene Understanding

## 2.1 Task Definition

The one-way light time between Earth and Mars varies from approximately 4 to 22 minutes, making continuous remote operations impracticable [1]. Decisions that must be taken while a rover traverses terrain not yet examined by ground operators therefore benefit from onboard perception and autonomous reasoning. The first branch of the proposed framework addresses this perception problem through semantic segmentation of rover-acquired colour images with red, green, and blue channels. More specifically, the experiments presented in this thesis use images captured by the Mast Camera (Mastcam) aboard the Curiosity rover, as provided by the Mars-Bench release of S5Mars [2, 3]. The objective is to convert each observation into a pixel-level representation of the surrounding scene that may support downstream functions such as terrain interpretation, traversability assessment, and route planning. The dataset was selected because its label space covers terrain elements relevant to environmental understanding while also including a *Rover* category, establishing a possible future interface with the damage-inspection branch.

A deep learning approach is adopted to compare architectures with different representational capacities and computational requirements. The objective is not to maximize segmentation accuracy in isolation, because potential onboard use makes computational efficiency a central design criterion, as also emphasized by previous research on lightweight semantic segmentation for Martian environments [4]. The experimental study therefore considers the accuracy-efficiency trade-off and subsequently investigates quantization as a means of reducing inference cost without introducing an unacceptable degradation in segmentation quality.

## 2.2 Dataset

The primary dataset adopted for the scene-understanding experiments is the Mars-Bench adaptation of $S^5$Mars. The original $S^5$Mars dataset was introduced by Zhang et al. for semi-supervised semantic segmentation of Martian scenes [2]. It

comprises 6,000 RGB images acquired by Mastcam aboard the Curiosity rover, each with a spatial resolution of 1200 × 1200 pixels. Its annotations follow a sparse, confidence-based protocol, where instead of assigning a category to every pixel, labels are retained only in regions for which the annotators considered the semantic interpretation sufficiently reliable. This design limits uncertain assignments at visually ambiguous boundaries, but also leaves part of each mask without a terrain label.

The Mars-Bench release was selected in preference to the original distribution to obtain a more reproducible and internally consistent evaluation protocol. Mars-Bench converts previously published Martian vision datasets into standardized formats and supplies predefined training, validation, and test partitions [3]. More importantly, its authors subjected the terrain-segmentation datasets to an additional expert review. They reported ambiguities among visually similar granular-material categories and consolidated the corresponding $S^5$Mars labels into a single *Sand/Soil* category. This refinement was intended to reduce label noise and make comparisons between models less dependent on uncertain distinctions. This Mars-Bench release used in this work contains 4,997 training images, 200 validation images, and 800 test images. Mars-Bench describes eight target categories, while the released masks expose an additional *Background* identifier. Consequently, the implementation uses nine mask identifiers: *Background*, *Bedrock*, *Hole*, *Ridge*, *Rock*, *Rover*, *Sand/Soil*, *Sky*, and *Track*

Two additional datasets included in Mars-Bench were used for a separate cross-dataset evaluation: MarsSeg MSL and MarsSeg MER [3]. Both originate from the Mars-Seg dataset family introduced by Li et al. [6] and were subsequently adapted to the standardized taxonomy and evaluation protocol defined by Mars-Bench. MarsSeg MSL comprises RGB Mastcam images acquired by the Curiosity rover during the Mars Science Laboratory (MSL) mission, whereas MarsSeg MER contains grayscale images captured by the Navigation Cameras (Navcams) and Panoramic Cameras (Pancams) aboard the Spirit and Opportunity rovers during the Mars Exploration Rover (MER) missions [6]. Their identical label mapping permits direct cross-mission evaluation without modifying the output layer or introducing an uncertain correspondence between categories. A model can be trained and selected using one mission and subsequently evaluated, without target-domain fine-tuning, on the test split of the other.

| Dataset | Rover(s) / Sensor(s) | Mask IDs | Training | Validation | Test |
|---|---|---|---|---|---|
| $S^5$Mars | Curiosity / Mastcam | 9 | 4,997 | 200 | 800 |
| MarsSeg MSL | Curiosity / Mastcam | 7 | 2,893 | 413 | 828 |
| MarsSeg MER | Spirit, Opportunity / Navcam, Pancam | 7 | 744 | 106 | 214 |

**Table 2.1.** Mars-Bench semantic segmentation datasets used in this work. The number of mask identifiers includes the *Background* label.

## 2.3 Pre-processing

A consistent input representation was enforced across all dataset splits. Each image was converted to RGB, ensuring a three-channel input irrespective of its original encoding, while the corresponding segmentation mask was retained as a single-channel categorical map. Image-mask pairs were subsequently resized to $512 \times 512$ pixels, a resolution selected to retain sufficient spatial detail for semantic segmentation. Bilinear interpolation was used for the RGB images, as it provides smooth resampling of continuous intensity values, while nearest-neighbour interpolation was applied to the segmentation masks to prevent the introduction of intermediate values that would not correspond to valid semantic categories.

Data augmentation was implemented as an optional training-only stage after resizing. The available transformations comprised horizontal flipping, vertical flipping and random 90° rotations with probability 0.5. When the rotation operation was triggered, the number of 90° rotations was sampled uniformly from $k \in \{0, 1, 2, 3\}$. The default configuration disabled augmentation, whereas the augmented configurations activated these transformations explicitly for comparative experiments. Following the optional augmentation stage, images were scaled to the $[0, 1]$ range and then channel-wise normalization was performed using the ImageNet mean values and standard deviations. This convention maintains compatibility with the ImageNet-pretrained encoders employed by several of the evaluated architectures. Segmentation masks were instead converted to integer tensors without normalization, thereby preserving their original class identifiers. No stochastic transformation was applied to the validation or test sets, as their samples underwent only the deterministic operations of RGB conversion, resizing, tensor conversion, and normalization, ensuring that the reported evaluation results were not affected by random data transformations.

## 2.4 Segmentation Architectures

The architecture set was selected with two complementary objectives: preserving continuity with the segmentation models considered in Mars-Bench and investigating a sufficiently broad accuracy-efficiency spectrum for potential onboard deployment. Mars-Bench includes U-Net, DeepLabV3+, SegFormer, and DPT among its semantic segmentation baselines [3]. Three of these, U-Net, DeepLabV3+, and SegFormer, were retained in this work. DeepLabV3 was added to examine an atrous-convolution architecture without the low-level feature decoder introduced in DeepLabV3+, whereas $\text{LCNet}_{3,7}$ and $\text{LCNet}_{3,11}$ were included as additional lightweight alternatives.

The resulting model pool comprises seven configurations and covers four substantially different design principles: skip-connected convolutional encoder-decoders, atrous spatial pyramid pooling, hierarchical Transformer representations, and partial-channel transformations. This diversity makes it possible to investigate whether the gains associated with increasingly expressive context modeling justify their computational cost.

For the S5Mars experiments, every architecture received a three-channel image resized to $512 \times 512$ pixels and produced nine channels of class scores at the same

spatial resolution, one for each class in the taxonomy. The U-Net, DeepLab, and SegFormer configurations returned logits without a final activation, whereas the project LCNet classifiers applied batch normalization and SELU after the final pointwise convolution. The training objective operated on these scores without an additional output softmax. All network components were left trainable.

The U-Net and DeepLab configurations were instantiated through the Segmentation Models PyTorch library [14]. Their encoders were initialized using weights learned on ImageNet, while their decoders and segmentation heads were initialized for the target task. SegFormer-B0 was instead initialized from a checkpoint fine-tuned for semantic segmentation on ADE20K [15], with its incompatible 150-class classifier replaced by a nine-channel layer, while the compatible encoder and decoding weights were retained. The two LCNet configurations were trained from random initialization, given the absence of pretrained weights. Consequently, comparisons among different model families capture the behavior of complete deployment candidates, including their initialization strategy, and should not be interpreted as controlled ablations of architecture alone.

### 2.4.1 U-Net Variants

U-Net is a convolutional neural network (CNN) with an encoder-decoder architecture originally proposed for biomedical image segmentation [7]. The contracting path reduces the spatial resolution of the input and extracts increasingly abstract representations, enabling the network to encode a wider image context. The expanding path restores a dense output at progressively higher resolutions. The defining characteristic of this architecture is the set of skip connections between corresponding encoder and decoder stages. These connections pass higher-resolution features directly to the decoder, where they can be combined with features that contain more context but less spatial detail. The decoder can therefore recover local information that may be weakened by repeated downsampling. This makes U-Net an appropriate baseline for terrain segmentation, where accurate class-boundary localization is important.

Both U-Net variants had a five-stage decoder where, at each stage, the decoder representation was upscaled using nearest-neighbor interpolation and concatenated with the corresponding encoder representation followed by two $3 \times 3$ convolutional blocks with batch normalization and ReLU activation. A final $3 \times 3$ segmentation head mapped the resulting 16-channel representation to the nine output classes.

**U-Net with ResNet-34.** The first variant makes use of ResNet-34 as the encoder. ResNets are a type of architecture that stacks the convolutional layers in blocks where the learned transformation of the block is added to the original block input via a shortcut connection [8]. Therefore, every block learns to correct the original representation rather than learning a completely new transformation. ResNet was proposed by He et al. as a way of solving the optimization degradation problem that is experienced when increasing the depth of the conventional convolutional networks. When the input and output dimensions agree, the identity shortcut introduces neither additional parameters nor a separate learned transformation.

In the case of U-Net, ResNet-34 provides a conventional hierarchical encoder whose intermediate representations can be transferred to the decoder at different

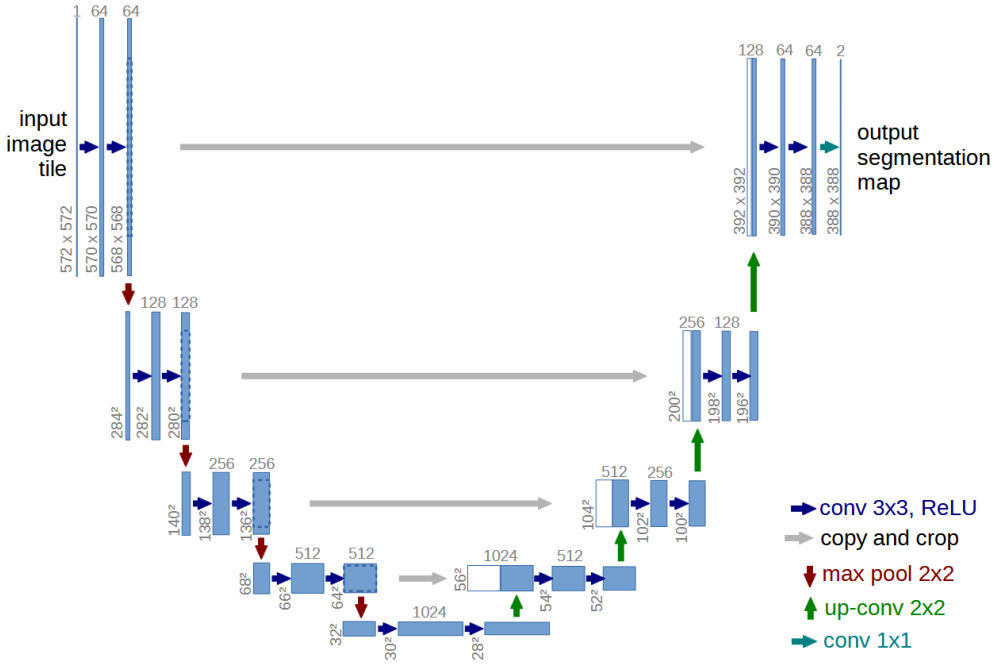



**Figure 2.1.** The original U-Net architecture proposed by Ronneberger et al., comprising a contracting path, an expanding path, and skip connections between corresponding resolution levels [7]. The implementations evaluated in this work replace the original contracting path with ResNet-34 or MobileNetV2.

spatial scales. The choice of this architecture was motivated by the fact that it was a higher-capacity convolutional variation in the U-Net family. It combines the established residual feature extractor with an explicit reconstruction path for dense prediction, while preserving architectural continuity with the U-Net baseline considered in Mars-Bench.

The same ResNet-34 encoder is also employed in the DeepLabV3 configuration described later in this section. Although the resulting networks are not identical outside the encoder, this shared component makes the comparison between skip-based reconstruction and atrous spatial pyramid pooling (ASPP) easier to interpret.

**U-Net with MobileNetV2.** The second variant replaces ResNet-34 with MobileNetV2, an architecture developed specifically for mobile and resource-constrained computer vision [9]. MobileNetV2 is an improved version of MobileNetV1, with the aim of optimizing accuracy while ensuring efficiency [18]. Its central component is the inverted residual block. Whereas the traditional residual block transforms the input representation in the middle of relatively wider representations, MobileNetV2 does this task in between narrower representations, known as bottlenecks. Each block first expands the input representation, performs spatial filtering through an inexpensive depthwise convolution, and then projects the result back to a lower-dimensional space. The final projection is linear, since Sandler et al. observed that applying a non-linearity within a narrow bottleneck can lose useful representational information.

Inside each inverted residual block, depthwise convolution applies a spatial filter independently to every expanded feature channel, whereas the preceding and subsequent pointwise convolutions expand, combine, and project the channel representations. By using spatial filtering and channel mixing in this manner, MobileNetV2 reduces the computational cost associated with typical dense convolutions, while residual connections preserve information between bottleneck representations whenever their spatial resolution and channel dimensionality remain unchanged [9].

The variant was therefore included to determine whether the U-Net topology could provide an effective segmentation model with a substantially lighter encoder. Its comparison with the ResNet-34 variant examines two deployment alternatives using the same architectural family: one centered on a conventional residual hierarchy and the other on mobile-oriented inverted residual blocks.

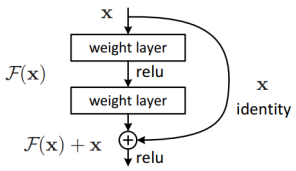
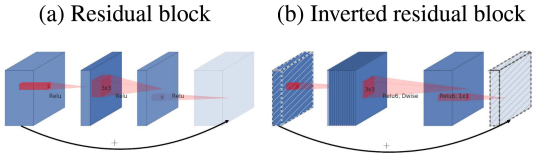



Figure 2. Residual learning: a building block.

**(a)** Residual learning block.




**(b)** Conventional and inverted residual blocks.

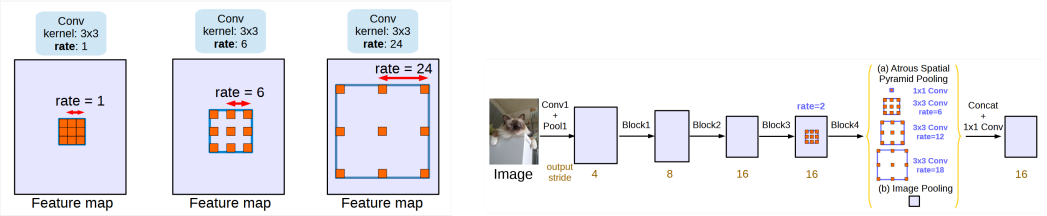
**Figure 2.2.** Original diagrams illustrating the principles underlying the two U-Net encoders. The residual learning block in (a) is reproduced from He et al. [8]; the comparison between conventional and inverted residual blocks in (b) is reproduced from Sandler et al. [9]. Block width in the MobileNetV2 diagram represents the relative number of channels.

### 2.4.2 DeepLab Variants

Semantic segmentation requires a model to preserve sufficiently detailed spatial information while also capturing the larger context needed to classify each image region. Conventional classification backbones gradually reduce feature-map resolution through strided convolutions and pooling, in a way that can discard spatial information that remains relevant to dense prediction. The DeepLab family addresses this aspect primarily through atrous convolution, also known as dilated convolution [10].

Atrous convolution introduces spacing between the positions sampled by a convolutional filter. Increasing this spacing enlarges the effective field of view without increasing the number of learned kernel parameters and it can produce denser final feature maps while retaining a broad receptive field [10]. The atrous rate determines the spacing, with a rate of one corresponding to conventional convolution. DeepLabV3 applies this operation within an ASPP module. ASPP sends the encoder output through several parallel branches whose different atrous rates capture context over different spatial extents. A separate image-pooling branch summarizes the complete feature map and supplies global contextual information. The outputs are then merged to form the representation used for pixel classification [10].

This multi-scale mechanism is relevant to Martian scene understanding, where terrain features exhibit substantial variations in spatial scale. Previous work has addressed this challenge by combining multi-scale feature extraction with the preservation of local details and small objects [17]. These considerations motivate the use of both broad contextual information and localized evidence in the present work.




**(a)** Atrous convolution at different sampling rates.

**(b)** ASPP augmented with image-level features.

**Figure 2.3.** Original diagrams illustrating the principal operations of DeepLabV3. Atrous convolution enlarges the effective field of view by changing the sampling rate, while ASPP combines parallel context branches and image-level pooling. Both diagrams are taken from Chen et al. [10].

**DeepLabV3 with ResNet-34.** The first variant uses ResNet-34 as its encoder. Unlike U-Net, DeepLabV3 does not use a multi-stage decoder with skip connections. Instead, the ASPP features are converted into class logits and resized to the input resolution using bilinear interpolation [10]. This variant was selected to compare two segmentation approaches built around the same backbone. U-Net–ResNet-34 progressively combines encoder and decoder features to recover spatial detail, whereas DeepLabV3 gathers multi-scale context at the final encoder stage. Other architectural differences mean that this comparison does not isolate the decoder alone, but it helps examine the strengths of each approach. DeepLabV3 also provides a reference for discussing DeepLabV3+, whose additional decoder uses low-level features to refine the segmentation boundaries.

**DeepLabV3+ with MobileNetV2.** DeepLabV3+ extends the predecessor by combining its multi-scale contextual encoder with a lightweight decoder [11]. The ASPP representation is first upsampled and then merged with an earlier feature map from the backbone, providing information that can support the refinement of object and region boundaries. The fused features are processed by the decoder before a final upsampling operation restores the prediction to the input resolution.

The resulting architecture combines two complementary mechanisms. Atrous convolution and ASPP encode contextual information over several spatial ranges, while the decoder reintroduces higher-resolution information that is less accessible from the deepest feature map alone. The original DeepLabV3+ study further employed depthwise-separable convolutions within the ASPP and decoder modules to reduce their computational cost [11].

The second variant makes use of the same MobileNetV2 backbone utilized by the previous U-Net configuration, providing a second cross-family point of comparison.

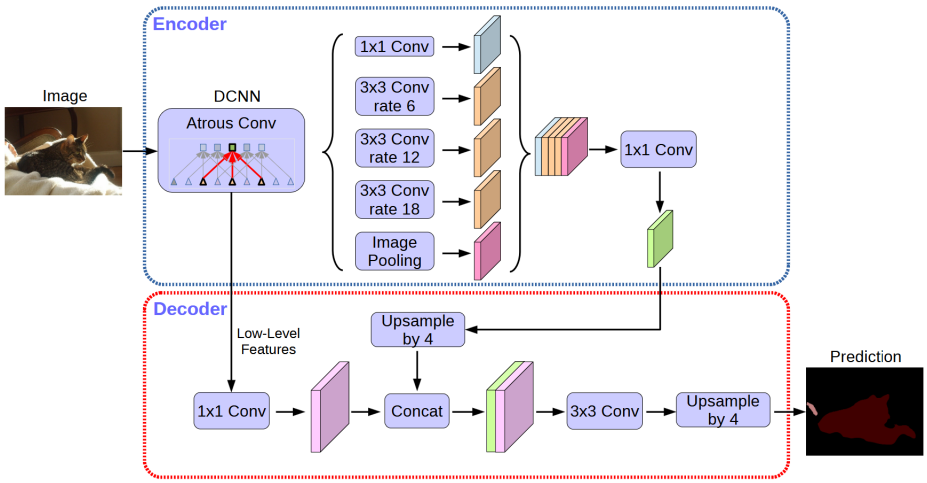



**Figure 2.4.** Original DeepLabV3+ architecture, Reproduced from Chen et al. [11].

This choice produces a deployment-oriented configuration of DeepLabV3+ rather than reproducing the modified Xception backbone emphasized in the original paper.

### 2.4.3 SegFormer-B0

SegFormer combines a hierarchical Transformer encoder, called Mix Transformer (MiT), with a lightweight multilayer perceptron (MLP) decoder [12]. The architecture was included to examine an attention-based alternative to the convolutional models described above.

Self-attention allows each position to gather information from distant parts of the image within a single layer, whereas local convolutions build larger context through successive layers [19]. For Martian semantic segmentation, this provides a reason to investigate whether relationships across an image can help discriminate terrain regions whose local appearance may be ambiguous. MiT extracts features in four stages, producing progressively larger representations. Overlapping patch embeddings preserve local continuity, while attention uses spatially reduced features to lower its computational cost. Each block also includes a Mix-FFN module, which adds local spatial information through a depthwise convolution. This removes the need for explicit positional embeddings and their interpolation when the input resolution changes [12].

The decoder projects the four encoder outputs into a common feature dimension, resizes them to the same spatial resolution, and fuses them before classification [12]. This brings all encoder stages together in a single fusion step, and its evaluation therefore examines whether this compact decoder can preserve the spatial detail needed for terrain segmentation.

SegFormer was included because it is one of the model families evaluated in Mars-Bench [3]. B0, its smallest variant, was selected to limit model size [12], while testing its accuracy-efficiency trade-off, with results presented in the evaluation chapter.

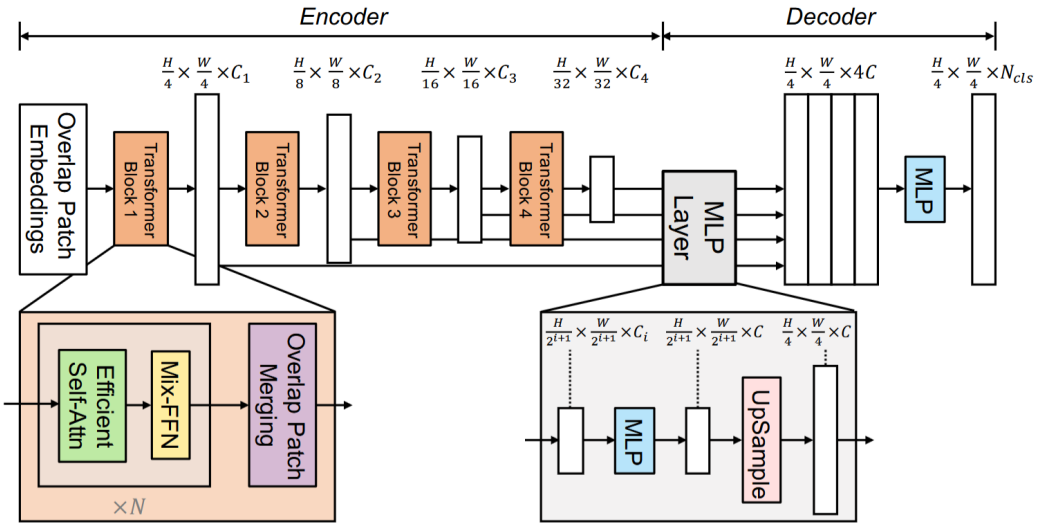



**Figure 2.5.** Original SegFormer architecture, showing the hierarchical Transformer encoder and the decoder that fuses its outputs. Reproduced from Xie et al. [12].

### 2.4.4 LCNet Variants

The Lightweight Context-Aware Network (LCNet) was introduced by Shi et al. to address the computational costs of semantic segmentation through partial-channel processing and a compact encoder-decoder design [13]. LCNet was included in this study to extend the architectural comparison towards models explicitly designed to limit computational cost, in addition to the architecture families considered in Mars-Bench. Its inclusion allows the study to investigate whether a smaller network could provide sufficient accuracy for Martian scene understanding.

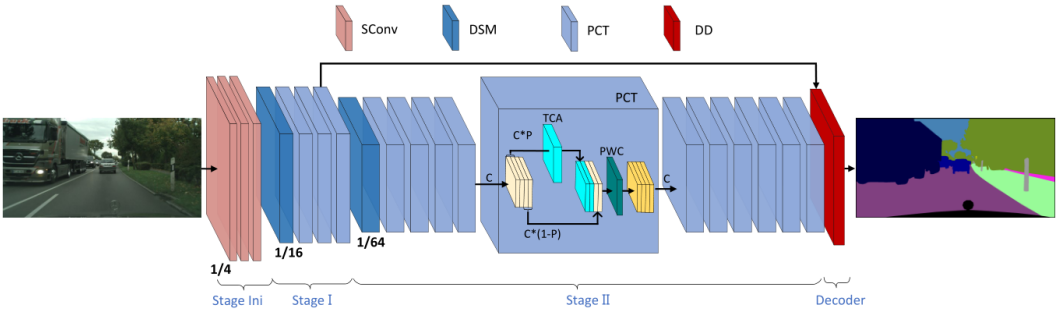



**Figure 2.6.** Overview of the LCNet architecture. Original diagram provided by the authors of [13].

Its main processing unit is the Partial-Channel Transformation (PCT) block, which splits the input features into two groups. One group passes directly to the output fusion, while the other undergoes context processing. The two groups are then concatenated and combined through a pointwise convolution [13]. In the project implementation, each path receives half of the channels, limiting the amount of spatial

processing while retaining information from both groups. The transformed group is processed by a Three-Branch Context Aggregation (TCA) module, where a standard convolution first produces intermediate features, which are then processed by local and dilated depthwise convolutions. Their outputs are added to the intermediate features, combining information from nearby positions and wider regions without further downsampling.

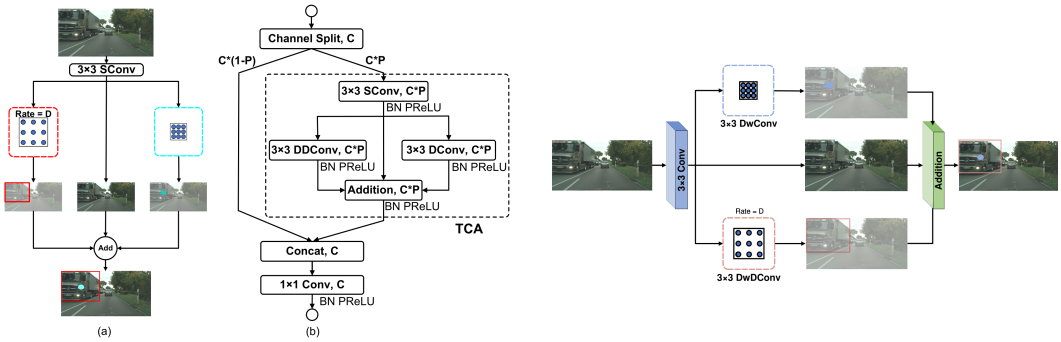



**Figure 2.7.** Partial-channel transformation (left) and three-branch context aggregation (right). Original diagrams provided by the authors of [13].

The encoder contains an initial convolutional stage followed by two PCT stages. Each PCT stage begins with a downsampling block that combines a strided convolution with max pooling, so most context processing takes place at reduced spatial resolutions. The decoder then brings together features from both stages through two attention mechanisms [13]. The shallower features provide spatial guidance, while the deeper features are refined through a reverse-guidance branch. After resizing the deeper features, the decoder combines them with the spatial guidance through element-wise multiplication. A depthwise convolution and a pointwise classifier produce the class scores, which are resized to the input resolution using bilinear interpolation.

Two configurations were implemented: $\text{LCNet}_{3,7}$ and $\text{LCNet}_{3,11}$. Both use three PCT blocks in the first encoder stage, followed by seven and eleven blocks in the second stage, respectively. $\text{LCNet}_{3,11}$ therefore adds four context-processing blocks without changing the overall design, allowing the comparison to examine whether a deeper encoder offers enough additional accuracy to justify its computational cost. Both variants were independently reimplemented for the project and configured to predict the nine S5Mars classes, using the random initialization described above.

### 2.4.5 Architectural Comparison

Table 2.2 compares the seven configurations independently of their predictive performance. Parameter counts refer to the nine-class implementations used for S5Mars and include all trainable layers. The reported 32-bit floating-point (FP32) storage is the theoretical memory required by the parameters alone and excludes intermediate activations, framework metadata, buffers, and optimizer state.

| Model | Encoder | Initialization | Params. (M) |
|---|---|---|---|
| U-Net | ResNet-34 | ImageNet | 24.438 |
| U-Net | MobileNetV2 | ImageNet | 6.630 |
| DeepLabV3 | ResNet-34 | ImageNet | 26.009 |
| DeepLabV3+ | MobileNetV2 | ImageNet | 4.381 |
| SegFormer-B0 | MiT-B0 | ADE20K | 3.716 |
| $\text{LCNet}_{3,7}$ | LCNet stem | Random | 0.509 |
| $\text{LCNet}_{3,11}$ | LCNet stem | Random | 0.729 |

**Table 2.2.** Comparison of the segmentation architectures implemented for the nine-class S5Mars task.

## 2.5 Training

The segmentation models share a configurable training pipeline that controls the optimization procedure, loss function, data sampling, and checkpoint selection. A common reference configuration provides shared optimization settings across the architecture families, while allowing individual components to be modified for the ablation studies. The U-Net, DeepLab, and SegFormer configurations were trained for 30 epochs, whereas the two LCNet configurations were trained for 60 epochs. Training used a batch size of 32, with images processed at the $(512 \times 512)$ resolution defined in the preprocessing section. The pretrained models are fine-tuned end to end, therefore all network parameters remain trainable throughout optimization, including the encoders initialized from pretrained weights, whereas the LCNet configurations are optimized from random initialization. The training images are shuffled at every epoch, while validation and test samples are processed without shuffling or stochastic augmentation.

The loss function combines pixel-wise cross-entropy (CE) with Generalized Dice Loss (GDL):

$$\mathcal{L} = \alpha \mathcal{L}_{\text{CE}} + (1 - \alpha)\mathcal{L}_{\text{GDL}}, \qquad \alpha = 0.5. \tag{2.1}$$

Cross-entropy supervises the probability assigned to the target class at each pixel, whereas the Dice component measures agreement between the predicted probability maps and the target regions. Generalized Dice introduces class-dependent weighting to account for differences in region size, following the formulation investigated by Sudre et al. for imbalanced segmentation tasks [20]. Combining the two terms therefore provides pixel-wise classification supervision alongside a region-overlap objective. This follows the rationale to combine the gradient stability of cross-entropy with the ability of Dice-based losses to handle class imbalance [23].

In the implemented Generalized Dice loss, class volumes are computed from the target masks across the current mini-batch. Classes present in the batch receive weights inversely proportional to the square of their pixel counts, while absent classes receive zero weight in the Dice calculation. These weights are recomputed for each

batch rather than derived from precomputed class frequency scores. A smoothing constant of ($10^{-5}$) is added to the numerator and denominator of the overlap ratio. The pipeline also supports a single loss function alone and alternative Dice weighting schemes. The available configurations include inverse-volume weighting or uniform weights over the classes present in the batch. These alternatives allow the contribution of the loss formulation to be examined separately from changes in architecture.

The Optimizer is AdamW [21], with a base learning rate of $10^{-4}$ and a weight-decay coefficient of $10^{-2}$. A linear warm-up during the first epoch is followed by cosine decay towards zero over the remaining epochs, without warm restarts [22]. Training runs for the fixed epoch budget without early stopping, while the checkpoint with the best validation performance is retained for evaluation. Training was performed on Kaggle using two NVIDIA T4 GPUs, each with 16 GB of VRAM.

**Training-data sampling and optional augmentation.** Standard sampling is the default configuration, while image oversampling is available as a separate training-only option. Unlike the class weighting inside Generalized Dice, which depends on pixel counts within a batch, oversampling uses the class frequency of the training images. It therefore modifies how frequently images are presented to the model without changing their masks or the loss definition. When oversampling is enabled, the number of training images containing each class is first computed. Each image receives the largest rarity factor among its eligible classes, where the rarity factor is the square root of the ratio between the most frequent class count and the count of the class under consideration. In the oversampling experiments reported in this thesis, the image weights were capped at 6 before normalization, and images were sampled with replacement. With the reference epoch-length multiplier of 1, the number of samples equals the number of training images, preserving the nominal training length although some images may be repeated and others omitted in an epoch. Geometric augmentation is controlled independently from sampling and remains disabled in the default configuration. When enabled, the transformations described in the preprocessing section are applied jointly to each training image and its mask. Neither augmentation nor oversampling modifies the validation or test distributions.

## 2.6 Evaluation Metrics

Segmentation performance is evaluated using pixel accuracy (PA), per-class Intersection over Union (IoU), and mean Intersection over Union (mIoU), following established semantic-segmentation evaluation practice [24]. Predictions are obtained by selecting the highest-scoring class at each pixel, and the metrics are computed from a confusion matrix accumulated over the entire evaluated partition rather than averaged across images or mini-batches.

Let $TP_c$, $FP_c$, and $FN_c$ denote the true-positive, false-positive, and false-negative

| Setting | Default configuration |
|---|---|
| Input resolution | $512 \times 512$ |
| Training epochs | 30 (60 for LCNet) |
| Batch size | 32 |
| Trainable parameters | Entire network |
| Optimizer | AdamW |
| Base learning rate | $10^{-4}$ |
| Weight decay | $10^{-2}$ |
| Learning-rate schedule | Linear warm-up and cosine decay |
| Warm-up duration | 1 epoch |
| Loss Function | Cross-entropy and Generalized Dice |
| Cross-entropy coefficient $\alpha$ | 0.5 |
| Dice class weighting | Inverse-square batch volume |
| Image-level oversampling | Disabled |
| Training augmentation | Disabled |
| Checkpoint selection | Highest validation mIoU |
| Random seed | 42 |

**Table 2.3.** Reference configuration for semantic-segmentation training. Experiment-specific changes to the loss, sampling, or augmentation settings are reported with the corresponding evaluations.

pixel counts for class $c$. The metrics are defined as

$$\begin{aligned} \text{Pixel Accuracy} &= \frac{\sum_c TP_c}{N}, \\ \text{IoU}_c &= \frac{TP_c}{TP_c + FP_c + FN_c}, \\ \text{mIoU} &= \frac{1}{|\mathcal{C}_+|} \sum_{c \in \mathcal{C}_+} \text{IoU}_c, \end{aligned} \tag{2.2}$$

where $N$ is the total number of evaluated pixels and $\mathcal{C}_+$ contains the classes present in the ground truth of the evaluated partition.

Pixel accuracy measures the overall proportion of correctly classified pixels, but consequently gives greater influence to classes occupying larger image regions. In contrast, mIoU assigns equal weight to each included class and serves as the primary checkpoint-selection metric. Per-class IoU complements this aggregate measure by exposing differences between semantic categories that a single score may conceal.

## 2.7 Optimization and Quantization

Quantization was investigated to test the efficiency of the segmentation models for resource-constrained deployment, with the objective of reducing model storage and inference costs while preserving segmentation accuracy. Selected FP32 checkpoints were exported to the Open Neural Network Exchange (ONNX) format as full models,

alongside FP16 variants obtained by converting eligible parameters and operations to half precision while maintaining FP32 inputs and outputs.

Static Post-Training Quantization (PTQ) to INT8 was applied without retraining the models. The PTQ configuration used entropy-based calibration with 500 images randomly sampled from the training set, without using their segmentation annotations. Selected weights and activations used symmetric quantization, with per-channel quantization for weights and per-tensor quantization for activations. The quantized models used the ONNX Quantize/Dequantize (QDQ) format, which introduces explicit nodes to represent conversions between floating point values and their quantized representations [25].

An alternative INT8 strategy used Quantization-Aware Training (QAT) with NVIDIA ModelOpt. Fake-quantization operations simulated rounding and clipping during fine-tuning, allowing the models to adapt to quantization effects [32]. The training objective combined the supervised segmentation loss with knowledge distillation from a frozen FP32 teacher, with coefficients of 0.7 and 0.3, respectively. Distillation used Kullback-Leibler divergence between softened class-probability maps, with the divergence scaled by the squared temperature. The resulting models were exported to ONNX with explicit quantization and dequantization operations, without an additional PTQ step.

The ModelOpt fine-tuning configuration used a maximum of 10 epochs, a mini-batch size of 2, AdamW with a learning rate of $10^{-5}$ and weight decay of $10^{-2}$, with one epoch of linear warm-up followed by cosine decay. The checkpoint with the highest validation mIoU was kept, with early stopping applied after three epochs without improvement.

Both INT8 strategies used selective quantization, keeping floating-point computations where required for compatibility or accuracy. For example, in the evaluated Conv-only configuration of SegFormer-B0, INT8 was restricted to convolutional layers, whereas linear projections, attention matrix multiplications, normalization, and softmax remained in floating point, with FP16 fallback in TensorRT. The resulting models therefore used mixed precision rather than an entirely integer computation graph. Quantization configurations were selected using the validation set, reserving the test set for final evaluation.

# Chapter 3

# Synthetic Dataset Generation

## 3.1 Design Requirement

The dataset was designed to contain RGB views covering all six wheel positions of a Curiosity Rover model, with one designated target wheel in each image. The anomaly class was restricted to localized through-holes in the wheel skin, with variations in shape, size, and position. Broken grousers and other failure modes remained outside the scope of this work. Normal samples were defined by the absence of an injected perforation so that dust and non-anomalous surface wear could remain part of the normal visual distribution.

The annotation requirements reflected the distinction between detecting damage and localizing it. Each sample required an image-level condition label and a target-wheel mask, while anomalous samples additionally required a binary mask identifying the damage region. The training split was required to contain only normal samples, whereas validation and test included both normal and damaged conditions. For evaluation, paired clean and damaged images were generated with the same camera, wheel pose, lighting, and surface-wear parameters, allowing direct comparisons in which the only difference was the introduced perforation. Each pair had to remain inside a single split to prevent paired scene leakage. Coverage of camera viewpoints, wheel rotations, illumination, and benign wear was required to avoid evaluating inspection under a single fixed appearance.

These requirements define a controlled synthetic benchmark for visible wheel perforations, used to evaluate how efficiently the proposed approaches tackle the task.

## 3.2 Assets preparation

Asset preparation focused on visually representing Curiosity and recreating its Martian surroundings as faithfully as possible, while keeping the scene suitable for controlled, wheel-specific modifications. The three-dimensional scene was assembled in Blender, which was used to prepare the rover and terrain assets and render the synthetic images.

**Rover model.** The rover asset was based on the publicly available Curiosity model distributed by NASA's Visualization Technology Applications and Development team [29]. Using an existing model avoided reconstructing the vehicle from photographs and provided both the wheel geometry and the surrounding rover structure. Retaining this context was important because the chassis and suspension contribute to the appearance of inspection images through occlusions and shadows.

After importing the model into Blender, the six wheels were separated into independently addressable objects, while the remaining vehicle structure was retained as an assembly. This organization enabled wheel-specific damage generation, rotation control, and mask extraction without rebuilding the rover geometry. The model was used for visual image generation rather than as an engineering representation of the rover's mechanical behaviour.

**Martian environment.** The terrain was derived from observations of Gale Crater acquired by the High Resolution Imaging Science Experiment (HiRISE), using a digital terrain model and associated orthorectified imagery [30]. These products provided a real observation-based foundation for the environment, including variations in elevation and surface appearance. A common region was extracted from both products, with the elevation data converted into a terrain mesh at metric scale and the corresponding imagery mapped onto its surface.

The orbital data were not detailed enough for close-up wheel views, so a local procedural terrain patch was added with grains, rocks, and small surface irregularities. Material effects provided finer visual detail without adding geometry, while rock models were reused to keep the scene manageable. These additions complemented the real terrain data but did not reproduce the exact ground surface at the site. Figure 3.1 shows the rover asset and the surrounding terrain assembled in Blender for synthetic image generation.

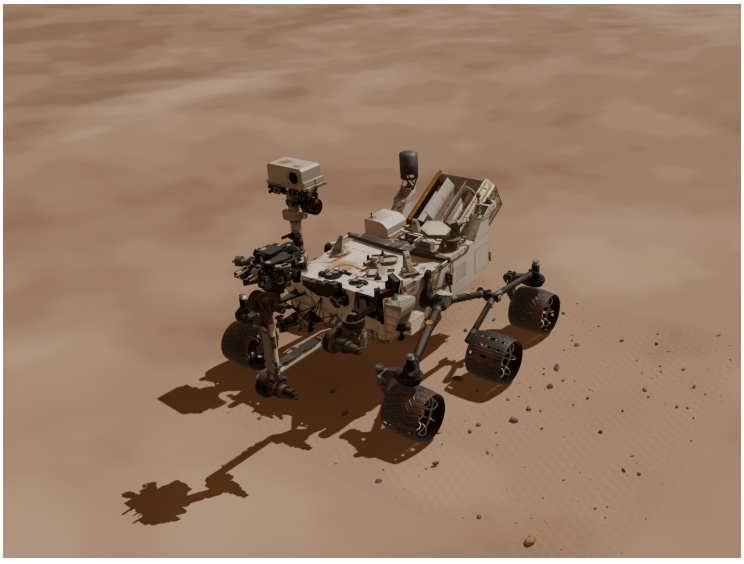
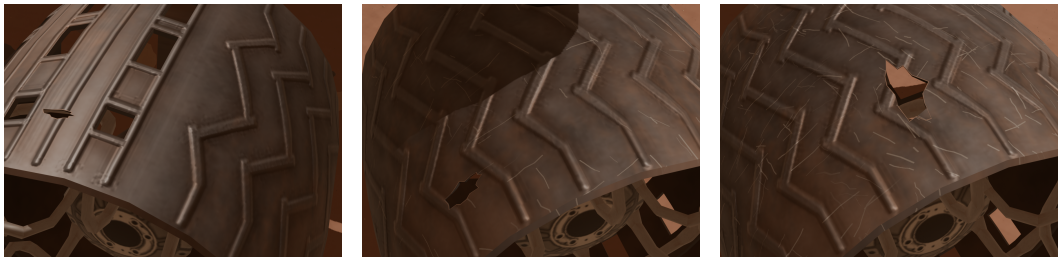


**Figure 3.1.** Overview of the Blender scene, showing the Curiosity rover asset and the surrounding terrain used for synthetic image generation.

## 3.3 Synthetic Damage Generation

Damage generation focused on a single anomaly class: through-holes in the wheel surface, with each anomalous image containing one injected hole on the designated target wheel. Hole contours were generated procedurally using three profile families: jagged slits, branched tears, and peeled-window profiles. Variations in contour irregularity, orientation, and dimensions provided different realizations within each family. Three size-based severity levels, denoted small, medium, and large, with their length sampled within 2.7-4.1 cm, 4.4-6.8 cm, and 7.3-11.6 cm, respectively. These categories describe the generated geometry rather than a mechanical assessment of wheel health. Figure 3.2 shows an example of each level.



(a) Small (b) Medium (c) Large

**Figure 3.2.** Examples of synthetic wheel damage at the three severity levels. The categories refer to the dimensions of the generated hole.

Each hole forms an opening through the wheel surface, allowing the terrain or other rover components behind it to be seen, depending on the viewpoint. Its appearance therefore also depends on the surrounding scene. The hole position varied across the tread and outer shoulder. Placement and wheel orientation were constrained so that the complete opening remained inside the image and was not hidden by the terrain or other rover components. For each damaged image, a corresponding binary mask was generated automatically, providing pixel-level annotations without manual labeling.

## 3.4 Domain Randomization

Domain randomization varies simulation conditions to expose models to a larger range of visual appearances [31]. In this work, variations were introduced during rendering to diversify normal wheel appearance and the context in which damage was observed, with the aim of reducing reliance on specific visual cues and helping the models distinguish actual damage from changes in appearance.

**Lighting and dust appearance.** Two rendering presets represented clear and dusty conditions (Figure 3.3). Alongside changes in illumination, the dusty preset introduced deposited surface dust and a mild adjustment to the camera response. Solar direction, direct and ambient illumination, and exposure were also perturbed within bounded ranges. These variations change shadows, highlights, and local con-

trast, requiring the detector to distinguish an opening in the wheel from appearance changes that can occur without damage.



(a) Clear preset (b) Dusty preset

**Figure 3.3.** Clear and dusty rendering presets applied to the same normal wheel, with identical camera pose and wheel orientation. The dusty preset also changes deposited dust and camera response, so the comparison does not isolate illumination alone.

**Normal surface variation.** Three surface conditions were considered: no wear, light wear, and evident wear (Figure 3.4). Additional scratches changed the surface appearance without creating holes or altering the wheel silhouette. Surface wear did not determine the anomaly label: a sample was anomalous only when a hole was introduced, and scratches were excluded from the anomaly mask. Including these patterns among normal appearances was intended to discourage false alarms caused by high-contrast marks or local texture changes, simulating real operational wear.



(a) No additional wear (b) Light wear (c) Evident wear

**Figure 3.4.** Normal surface conditions under identical lighting, camera pose, and wheel orientation. All three examples contain no injected hole and belong to the normal class.

**Wheel selection and rotation.** The target wheel was varied across all six rover wheels, introducing different surrounding suspension and chassis structures. For standalone normal samples, a shared rotation was applied to all six wheels using eight phases separated by 45°. Rotation changes the visible arrangement of tread features and surface marks, reducing dependence on a fixed angular configuration. For

damaged samples, wheel orientation was constrained by the visibility requirements described in Section 3.3.

**Procedural ground detail.** The local terrain incorporated procedurally generated relief, grains, rock fragments, and fine surface texture. These features introduce background clutter and cast shadows near the wheel, which must be distinguished from the target damage. The generated terrain realization was kept throughout dataset rendering rather than regenerated for each sample, for efficiency purposes. Its projected appearance nevertheless varied with camera pose and illumination.

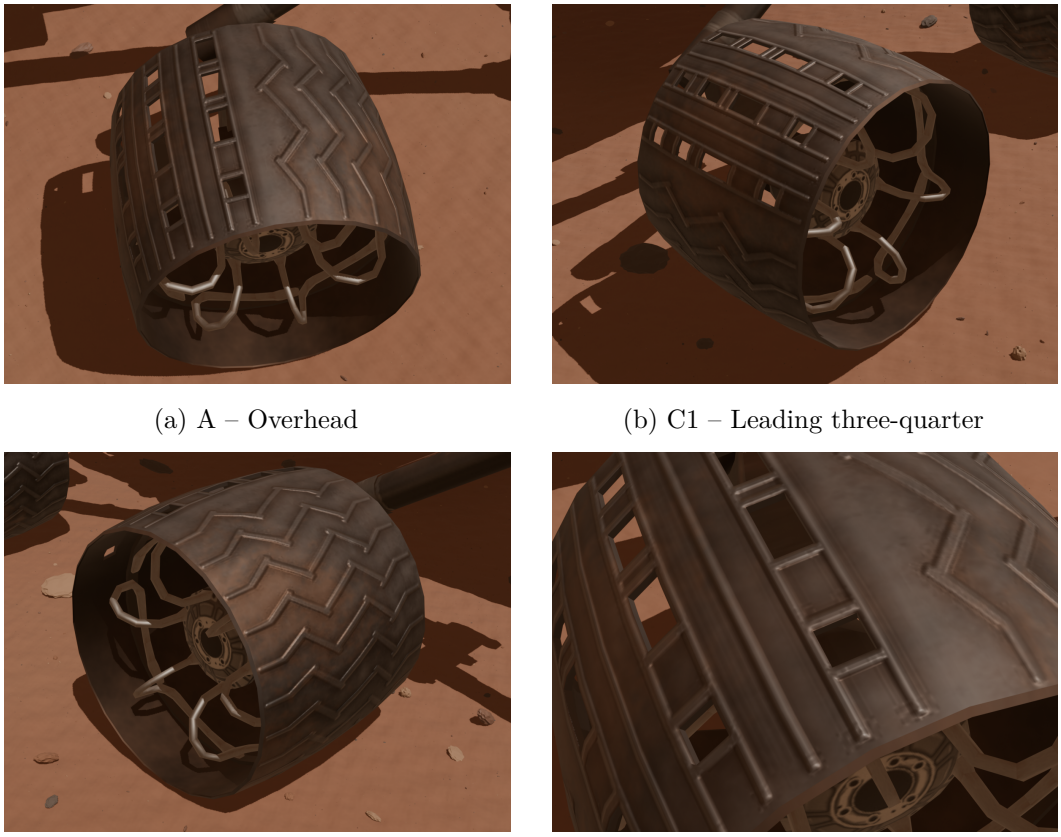
**Camera poses.** Four wheel-centric camera configurations were used: an overhead view (A), leading and trailing three-quarter views (C1 and C2), and an upper detail view (D), as shown in Figure 3.5. The first three configurations included the complete wheel, whereas D intentionally captured a closer, partial view. Small perturbations in camera position, viewing direction, and focal length provided variation within each configuration. Changing viewpoint modifies perspective, the visible wheel surface, and the apparent size of a hole. Whole-wheel views retain more context but represent small defects with fewer pixels, while the detail view magnifies local features. The different camera configurations were also included to compare detection and localization performance across viewpoints and identify the pose best suited to wheel-damage inspection.

Within each clean/hole pair, lighting, surface wear, camera settings, and wheel orientation were kept identical. Only the injected hole distinguished the two scene states, avoiding appearance differences that could otherwise provide unintended label cues within a pair.

## 3.5 Quality Assurance

Quality assurance combined automated checks with visual inspection of selected renders. Geometric and annotation constraints determined sample acceptance, while contrast diagnostics identified potentially challenging examples without automatically excluding them. The target wheel was checked for appropriate framing and unwanted occlusions. Full-wheel views required the wheel to remain within the image, whereas the partial framing of pose D was intentional and therefore admissible. For anomalous samples, the opening and its complete boundary had to remain visible. Additional checks verified wheel-ground contact and ensured that the ground region covered by the camera remained inside the detailed terrain patch, avoiding exposed patch boundaries or unsupported wheel placement.

Damage masks were required to be spatially consistent with the target-wheel region, and free of random connected components, while severity-dependent thresholds prevented annotations from becoming excessively small. Matched normal and anomalous images were compared to assess the appearance change produced by the anomalous detail and low contrast cases were kept and flagged, provided that the remaining acceptance checks were satisfied.



(a) A – Overhead

(b) C1 – Leading three-quarter

(c) C2 – Trailing three-quarter

(d) D – Upper detail

**Figure 3.5.** The four camera configurations shown for the same normal wheel under the dusty preset, without additional wear. Wheel orientation is unchanged across views.

## 3.6 Final Dataset Composition

The final dataset contains 10,000 RGB images, comprising 7,500 normal samples and 1,250 matched normal-anomalous pairs. This results in 8,750 normal images and 1,250 anomalous images, each containing one injected hole. Every image is associated with a target-wheel mask, while anomalous images additionally include a binary damage mask.

The training set contains only normal images, allowing the anomaly-detection models to learn wheel appearance without exposure to injected defects. Validation and test sets include both conditions, as summarized in Table 3.1. Validation contains 500 standalone normal images and 250 pairs, whereas the test set consists of 1,000 pairs. Split assignments were established before rendering, and each pair was treated as an indivisible unit, meaning that an anomalous image and its normal counterpart could not be assigned to different subsets.

The main category distributions are reported in Table 3.2. The six rover wheels are represented approximately uniformly, with 1,666–1,667 images per wheel. Defect severity and placement distributions refer only to anomalous images, with large defects restricted to the tread.

| Split | Normal | Anomalous | Total |
|---|---|---|---|
| Training | 7,000 | 0 | 7,000 |
| Validation | 750 | 250 | 1,000 |
| Test | 1,000 | 1,000 | 2,000 |
| Total | 8,750 | 1,250 | 10,000 |

**Table 3.1.** Composition of the synthetic dataset across training, validation, and test splits.

| Factor | Categories | Share (%) |
|---|---|---|
| Camera pose | A / C1 / C2 / D | 40 / 27.5 / 27.5 / 5 |
| Lighting | Dusty / clear | 75 / 25 |
| Surface wear | Current / light / evident | 20 / 45 / 35 |
| Defect severity | Small / medium / large | 50 / 40 / 10 |
| Defect location | Tread / outer shoulder | 80 / 20 |

**Table 3.2.** Main category distributions in the final dataset. Camera pose, lighting, and surface wear percentages refer to all 10,000 images; defect severity and location percentages refer to the 1,250 anomalous images.

Images were generated through an automated batch pipeline in Blender 5.2 using Eevee, at a resolution of $1200 \times 900$ pixels with 64 rendering samples, and saved as 8-bit RGB PNG files. For each sample, the prepared scene was restored to its reference state before applying the parameters specified by the generation plan. Standalone normal samples required one render, whereas paired samples required a normal and an anomalous render with matching camera, illumination, surface appearance, and wheel configuration. Annotation information was obtained alongside the RGB renders, and the normal wheel mask was kept for both members of each pair, for possible future use.

# Chapter 4

# Synthetic Data-Driven Damage Inspection

## 4.1 Task Definition

The second branch of the proposed framework addresses visual wheel inspection through the detection and localization of damage in RGB images. The experiments use the synthetic Curiosity wheel dataset introduced in the previous chapter, with localized holes as the target anomaly class. The objective is both to distinguish normal from damaged observations and to identify the image regions associated with the damage. Accordingly, the methods produce an image anomaly score and a spatial anomaly map, allowing detection and localization to be examined separately.

The problem was approached through visual anomaly detection using a normal-only training protocol, consistent with the setting adopted by benchmarks such as MVTec AD [33]. Only normal images from the generated dataset were used for training, while the rendered damaged images and their annotations were reserved for validation and testing. This choice was motivated by the aim of reducing dependence on annotated damage examples during training, rather than fitting a supervised segmentation model directly to the available hole masks. Normality was defined by the absence of the modeled perforations, not by a clear wheel surface, so that other appearance variations remained part of the normal distribution.

Several deep learning-based anomaly detection approaches were investigated to compare their ability to identify localized defects without confusing them with the appearance changes introduced during dataset generation. Compact configurations were also considered in view of the longer-term possibility of onboard inspection. The resulting study provides a controlled assessment of damage detection and localization within the synthetic domain, while validation on real rover imagery remains outside the scope of the present experiments.

## 4.2 Pre-processing

All images were loaded as three-channel RGB observations and converted to floating-point tensors. Their pixel intensities were rescaled from the original 8-bit representation to the $[0, 1]$ interval before any model-specific normalization. The experiments

included full frame inputs and wheel focused configurations using fixed square crops for each camera pose. The crop windows were defined from normal validation observations and applied consistently across dataset splits, before resizing. At inference, the pose identifier determined the crop, without requiring the damage mask. The already detailed framing of pose D did not receive an additional pose-specific crop. Wheel focused framing was motivated by the aim of dedicating more of the available input resolution to the inspected rover part. The intention was to keep useful surface detail at moderate network input sizes, favouring a controlled camera configuration that centers the target wheel. Figure 4.1 illustrates this preparation for pose A. In a future integration of the two framework branches, the *Rover* category predicted by the scene understanding module could help identify vehicle regions and help further wheel-localization steps. This connection was not implemented in the present work, as the segmentation label does not directly identify individual wheels.

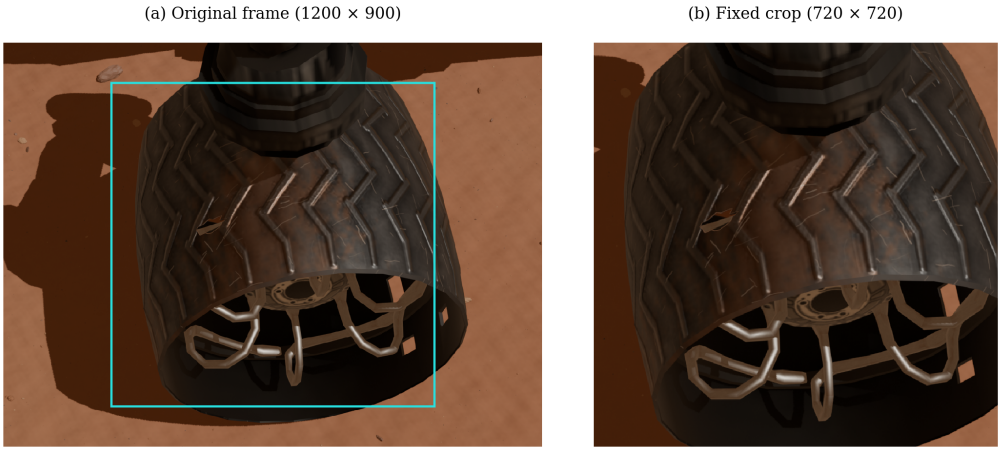



**Figure 4.1.** Comparison before and after the fixed crop for camera pose A. (a) Original $1200 \times 900$-pixel observation, with the crop window highlighted in cyan. (b) The retained $720 \times 720$-pixel region, shown before resizing to the network input resolution. Both panels depict the same dataset image.

Input resolution depended on the model and experimental configuration. It was treated as a compromise between computational cost and the spatial detail available for inspection, because strong downsampling can leave small defects represented by very few pixels, while larger inputs increase processing and memory requirements. The effects of these choices are examined in the evaluation chapter.

## 4.3 Anomaly Detection Models

The model set was selected with two complementary objectives: comparing different anomaly detection strategies compatible with the normal-only training strategy and investigating their computational requirements for potential onboard inspection. The resulting pool comprises four model families: PatchCore [34], EfficientAD-S [37], SuperSimpleNet [40], and TinyGLASS [41]. These methods cover three main

principles: matching against stored normal features, measuring student-teacher discrepancies, and learning to discriminate normal observations from synthetic perturbations. In the latter case, anomalous training examples are created from clean inputs in feature or image space, maintaining the separation from the damaged renders reserved for validation and testing.

The selection also contrasts the storage and search costs of a descriptor memory with the feature-extraction and prediction costs of feed-forward networks. PatchCore provides a memory-based reference, whereas the other three models compute anomaly scores through learned network outputs. EfficientAD-S and the ResNet-18 configuration of TinyGLASS represent compact alternatives, while SuperSimpleNet uses a larger Wide ResNet-50-2 feature extractor. This diversity makes it possible to examine the trade-off between damage detection and localization quality, memory requirements, and inference latency, and to assess whether more costly representations help distinguish localized perforations from normal appearance variations.

### 4.3.1 PatchCore

PatchCore detects anomalies by comparing local image features with a memory of defect-free examples, rather than learning predefined damage categories [34]. A convolutional backbone pretrained on ImageNet remains frozen, with features from two intermediate stages aggregated over local neighbourhoods and combined across scales, preserving local detail while incorporating spatial context.

During training, patch descriptors extracted from normal images are collected into a memory bank, from which greedy *coreset* selection retains a representative subset to limit redundancy and reduce the cost of nearest-neighbour searches. At inference, each test patch is compared with the stored descriptors and assigned an anomaly score equal to its Euclidean distance from the nearest neighbour, with the resulting scores mapped back to their spatial positions to form an anomaly map. The image score is based on the most anomalous patch, although the original formulation also applies neighbourhood-based reweighting when computing this score [34]. Figure 4.2 illustrates this process, from memory construction to patch and image anomaly scoring.

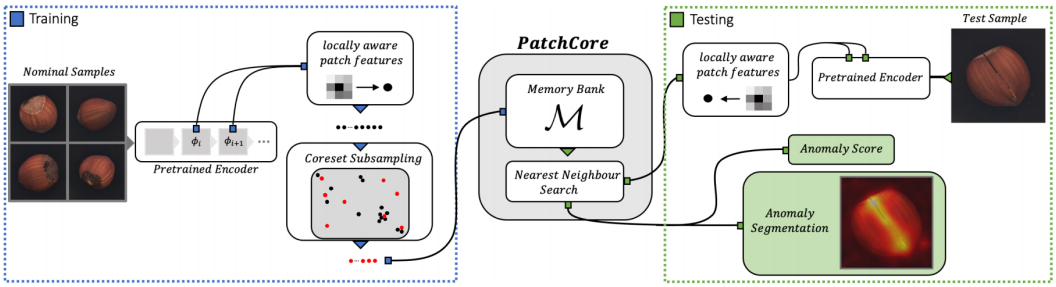



**Figure 4.2.** Original overview of PatchCore: normal-feature memory construction on the left and nearest-neighbour-based anomaly detection and localization on the right. Reproduced from Roth et al. [34].

PatchCore was included as a memory-based baseline compatible with normal-only training. Its local scoring mechanism is relevant to wheel perforations, which

affect limited image regions, while providing both detection and localization outputs. Related applications include BladeSynth, where PatchCore was fitted using normal rendered aero-engine blade images [35], and surface inspection of shock absorber connecting rods, where both the original method and an enhanced variant were evaluated [36]. These studies provide precedents for metallic component inspection, without establishing performance on Martian rover imagery.

In this project, a ResNet-18 configuration, denoted PatchCore Small (PC-S), was considered to reduce feature-extraction cost, alongside PatchCore Large (PC-L), which uses Wide ResNet-50-2. Both configurations use frozen pretrained features. To bound memory construction, patches are randomly sampled from each clean image and kept in a capped candidate reservoir before greedy coreset selection. Consequently, the coreset is selected from bounded candidates rather than from every extracted training patch. This introduces a trade-off between computational cost and coverage of normal appearances. Memory bank size also influences storage requirements and nearest-neighbour inference cost, making it an important factor in the efficiency assessment.

### 4.3.2 EfficientAD

EfficientAD detects deviations from normal appearance through a student-teacher model, in which a student learns to reproduce a frozen teacher's features using clean images [37]. The teacher is a compact Patch Description Network (PDN), distilled from a deeper pretrained network to avoid executing that larger backbone at inference. Differences between teacher and student outputs provide a local anomaly map, where larger discrepancies indicate possible departures from the normal training distribution.

Because local descriptors may appear normal even when objects are arranged incorrectly, EfficientAD also includes an autoencoder that reconstructs teacher features from the complete image. An additional student output learns the autoencoder's normal reconstruction behaviour, allowing the global anomaly map to measure their disagreement while accounting for systematic reconstruction errors. The two maps are calibrated and averaged [37], as illustrated in Figure 4.3.

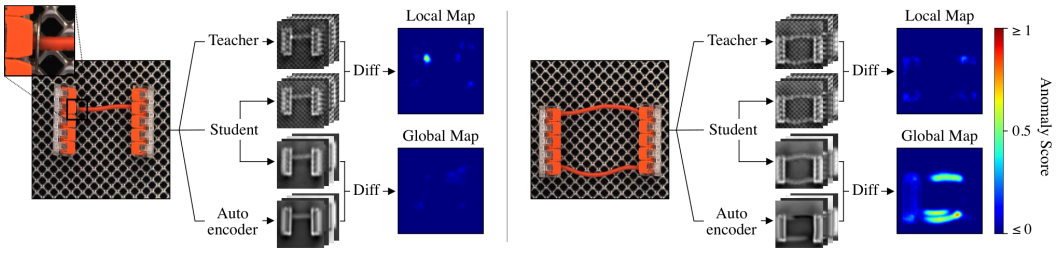



**Figure 4.3.** EfficientAD illustrated on a structural anomaly (left) and a logical anomaly (right), showing the local and global detection branches. Reproduced from Batzner et al. [37].

EfficientAD was included to investigate an alternative to PatchCore's memory-based matching, since its inference relies on learned network outputs rather than a nearest-neighbour search over stored descriptors. A related application is auto-

motive inspection on AutoVI, where the authors evaluated EfficientAD alongside other detectors on assembly-line images with variable illumination and cluttered backgrounds [38]. Although these experiments concern assembly defects rather than wheel perforations, they provide a relevant precedent for evaluating the method on mechanical components under variable acquisition conditions.

The experiments used EfficientAD-S, keeping a pretrained teacher while optimizing the student and autoencoder on clean wheel images. Training emphasized the largest feature discrepancies through hard-feature mining and included an auxiliary penalty on ImageNet images to discourage imitation beyond normal wheel appearance. In the global baseline configuration, calibration used only clean validation images, and the maximum of the combined anomaly map provided the image score.

### 4.3.3 SuperSimpleNet

SuperSimpleNet learns to distinguish normal image features from artificially perturbed ones, allowing a discriminative detector to be trained without examples of actual damage [39]. Rather than modifying the input images, it adds Gaussian noise to feature regions selected by thresholded Perlin noise, producing spatially coherent perturbations whose locations provide the training labels. Although the method can also exploit annotated defects, this study considers its clean-only setting.

A pretrained convolutional backbone extracts intermediate features, which are aligned to a common spatial resolution and pooled to incorporate neighboring context. A trainable adaptor prepares these features for a segmentation head, while a separate classification head combines the predicted anomaly map with image features to estimate an image score. In the extended formulation, the classification branch receives the unadapted features, allowing the two heads to operate on different representations [40]. Figure 4.4 illustrates this architecture, including the anomaly generation mechanism that is active during training but bypassed at inference.

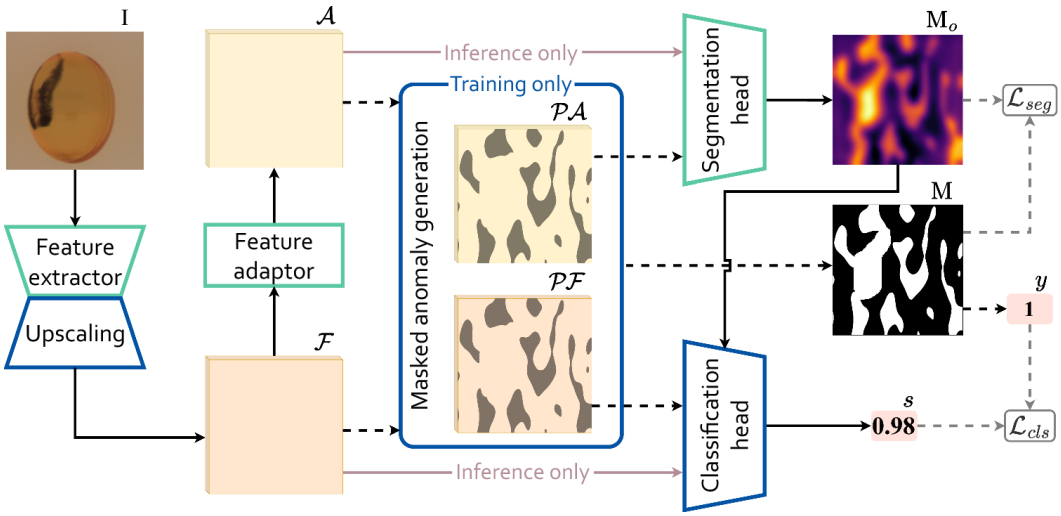



**Figure 4.4.** Architecture of the extended SuperSimpleNet formulation. Synthetic feature perturbations are introduced during training, whereas inference directly produces an anomaly map and a classification score. Reproduced without modification from Rolih et al. [40].

SuperSimpleNet was included to compare feature-space anomaly synthesis with PatchCore's memory-based matching and EfficientAD's student-teacher formulation. This comparison investigates whether learning from localized feature perturbations can support the detection of wheel perforations without using damaged renders for weight optimization. Its feed-forward inference also avoids searching a stored descriptor bank, although the computational cost of the pretrained backbone remains relevant. A related application is the normal-only inspection of automotive assemblies on AutoVI, where SuperSimpleNet was evaluated alongside other anomaly detectors [38].

The project implementation uses a frozen Wide ResNet-50-2 pretrained on ImageNet, extracting features from its second and third residual stages while optimizing the adaptor and prediction heads on clean wheel images. The generated perturbations are distinct from the holes rendered in Blender and do not require their annotations. Besides the original classification-head score, the experiments considered an image score obtained by averaging the largest 1% of the raw anomaly-map values on the native feature grid, before resizing and smoothing. Restricting synthetic perturbations to the target-wheel region was also investigated as an ablation, using the wheel mask during training only.

### 4.3.4 TinyGLASS

TinyGLASS is a compact adaptation of GLASS that learns to separate normal features from synthetic anomalies, replacing the original Wide ResNet backbone with ResNet-18 to reduce feature-extraction cost [41]. Intermediate features are combined across scales and processed by a patch-wise discriminator, which assigns higher scores to regions that depart from the normal training distribution.

Training combines the two complementary synthesis strategies introduced by GLASS. Local Anomaly Synthesis (LAS) creates altered image regions by blending external textures within randomized masks, whereas Global Anomaly Synthesis (GAS) perturbs normal features with Gaussian noise and refines them through gradient ascent. These gradient-guided perturbations provide difficult negative examples, while projection constraints limit their distance from normal features [42]. Combining image alterations with feature perturbations exposes the discriminator to both visible appearance changes and deviations close to normality. The synthesis mechanisms are used only during training, leaving a feed-forward prediction path at inference, as illustrated in Figure 4.5.

TinyGLASS was selected to examine the accuracy-efficiency trade-off of a compact synthesis-based detector. In the original study, the method was evaluated on the custom M&Ms candy dataset (MMS), including a crack-hole category, and deployed on the Sony IMX500 sensor [41], providing a related example of localized defect inspection.

The baseline implementation kept the ImageNet-pretrained ResNet-18 frozen and used fixed patch pooling for feature compression, without the separate trainable adaptor shown in the original figure. Alongside texture-based LAS, a task-specific variant generated irregular, darkened regions inside the target wheel to approximate the appearance of perforations. These two-dimensional alterations were created from clean renders and their wheel masks, rather than from the damaged Blender

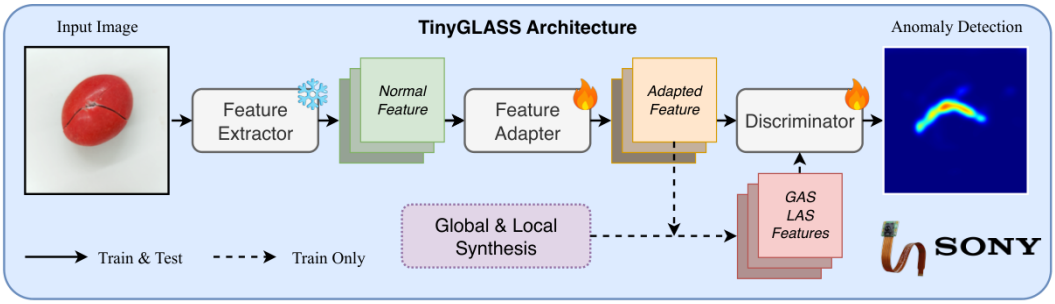



**Figure 4.5.** Original TinyGLASS architecture, showing the feature-processing path and the anomaly synthesis mechanisms used during training only. Reproduced without modification from Bonazzi et al. [41], under the CC BY 4.0 license.

images, and required no masks during inference. Feature resolution was also varied to examine the balance between computational cost and localization detail. The predicted anomaly maps were resized and smoothed, whereas the image score was obtained from the maximum patch score before this post processing.

## 4.4 Evaluation Metrics

Performance is evaluated at two levels: image detection, which distinguishes clean images from images containing anomalous wheel damage, and pixel localization, which compares the anomaly maps with the ground-truth hole masks. Both levels are assessed using the area under the receiver operating characteristic curve (AUROC) and Average Precision (AP), without selecting a fixed decision threshold.

These metrics are defined as

$$\begin{aligned} \mathrm{AUROC} &= \int_0^1 \mathrm{TPR}(u)\,\mathrm{d}u, \\ \mathrm{AP} &= \sum_j (R_j - R_{j-1}) P_j, \end{aligned} \tag{4.1}$$

where $\mathrm{TPR}(u)$ is the true-positive rate at false-positive rate (FPR) $u$, while $P_j$ and $R_j$ denote precision and recall at successive decreasing score thresholds, with $R_0 = 0$. Scores and labels are accumulated over the entire evaluated partition rather than averaging metrics across individual images.

AUROC describes the ability to rank anomalous images or pixels above normal ones, whereas AP summarizes the precision achieved as recall increases. AP is particularly relevant to localization because defect pixels occupy a small proportion of the input, so even a low false positive rate can produce many incorrect detections when normal pixels dominate [43]. Since AP also depends on anomaly prevalence, comparisons between configurations must account for differences in the evaluated data.

Localization is also examined using Per-Region Overlap (PRO), which measures the fraction of detected pixels within each connected ground truth anomalous region and averages these fractions with equal weight. This prevents larger defects from dominating the evaluation just because of their area [44]. The normalized area under

the PRO curve (AUPRO) is reported up to false positive rates of 0.05, 0.10, and 0.30, with the stricter limits emphasizing anomalous detection when fewer false detections are permitted. Computational efficiency is assessed separately through inference latency and GPU memory measurements, with the measurement conditions specified in the evaluation chapter.

# Chapter 5

# Evaluation of Lightweight Scene Understanding

## 5.1 Experimental Setup

Deployment benchmarks were run on a single NVIDIA T4 GPU in the Kaggle environment. Latency was averaged over 50 forward passes after 10 warm-up runs, with synchronization ensuring that GPU execution had completed before each measurement ended. Energy and peak memory were measured in a separate loop that continued for at least 100 inference runs and 30 seconds. GPU power and memory were sampled every 20 ms through the NVIDIA Management Library (NVML), and energy per image was obtained by dividing the estimated GPU energy consumed during the loop by the number of processed images.

## 5.2 Segmentation Accuracy

The various architectures were evaluated on the S5Mars test split, using the checkpoint with the highest validation mIoU for each training run. The U-Net, DeepLab, and SegFormer followed the training procedure described in Chapter 2, using 30 epochs for training, whereas the two LCNet variants were trained for 60 epochs, without data augmentation or oversampling. Table 5.1 reports the results obtained, the validation mIoU used for checkpoint selection alongside the test mIoU and pixel accuracy, keeping model selection separate from the final evaluation.

SegFormer-B0 achieved the highest test mIoU, 75.55%, followed by DeepLabV3 with ResNet34 at 73.79%. Its accuracy is particularly relevant to the lightweight objective: with 3.72 million parameters, compared with 26.01 million for DeepLabV3, SegFormer-B0 obtained a higher mIoU with approximately 86% fewer parameters (Table 2.2). U-Net with ResNet34 achieved the highest pixel accuracy, 91.73%, whereas SegFormer-B0 led on average IoU.

U-Net with MobileNetV2 retained essentially the same test mIoU as its ResNet34 counterpart, 64.57% versus 64.55%, while reducing the parameter count by approximately 73%. The LCNet variants explored a much smaller number of parameters: $\text{LCNet}_{3,7}$ and $\text{LCNet}_{3,11}$ reached 57.89% and 59.11% mIoU with only 0.51 and 0.73 million parameters, respectively. Together, these results show the available trade-off

| Configuration | Val mIoU | Test mIoU | Test PA |
|---|---|---|---|
| U-Net (R34) | 81.52 | 64.55 | 91.73 |
| U-Net (MV2) | 71.98 | 64.57 | 91.22 |
| DeepLabV3 (R34) | 83.14 | 73.79 | 91.63 |
| DeepLabV3+ (MV2) | 80.81 | 64.07 | 90.59 |
| SegFormer-B0 | 80.28 | 75.55 | 91.27 |
| $LCNet_{3,7}$ | 60.59 | 57.89 | 88.87 |
| $LCNet_{3,11}$ | 64.54 | 59.11 | 88.98 |

**Table 5.1.** S5Mars accuracy of the evaluated FP32 configurations. R34 and MV2 denote ResNet34 and MobileNetV2, respectively. Validation mIoU identifies the selected checkpoint, while test metrics describe its held-out performance.

between class-averaged accuracy and parameter storage, from the higher accuracy SegFormer-B0 configuration to the sub-million-parameter LCNet models.

The ranking also differs from the aggregate Mars-Bench results, where U-Net was the strongest segmentation baseline under frozen backbone feature extraction [3]. Here, SegFormer-B0 leads the S5Mars comparison after end-to-end fine-tuning, while the MobileNetV2 and LCNet configurations extend the study towards lower parameter budgets. Therefore Mars-Bench provides context for the architectural comparison, even though differences in training procedure prevent a direct ranking across the two studies.

Table 5.2 reports test IoU for the target class categories. The aggregate mIoU corresponds to the mean over these categories, while the additional *Background* identifier is not included in the table, given its absence in the ground truth labels. The definitions and aggregation procedure are given in Section 2.6.

| Model | Bedrock | Hole | Ridge | Rock | Rover | Sand / Soil | Sky | Track |
|---|---|---|---|---|---|---|---|---|
| U-Net (R34) | 73.32 | 9.75 | 92.28 | 26.47 | 67.99 | 89.95 | 94.56 | 62.07 |
| U-Net (MV2) | 71.60 | 0.00 | 92.17 | 21.58 | 82.52 | 89.34 | 94.26 | 65.11 |
| DeepLabV3 (R34) | 73.28 | 56.70 | 91.91 | 27.81 | 84.27 | 89.69 | 94.04 | 72.61 |
| DeepLabV3+ (MV2) | 70.55 | 9.65 | 91.89 | 21.97 | 84.99 | 88.39 | 94.50 | 50.59 |
| SegFormer-B0 | 71.85 | 78.11 | 90.20 | 21.77 | 83.03 | 89.48 | 94.02 | 75.96 |
| $LCNet_{3,7}$ | 67.08 | 9.26 | 90.42 | 17.18 | 73.72 | 86.45 | 92.95 | 26.09 |
| $LCNet_{3,11}$ | 67.42 | 9.49 | 90.77 | 18.85 | 72.48 | 86.60 | 93.62 | 33.63 |

**Table 5.2.** Per-class IoU on the S5Mars test split. Model abbreviations and selected checkpoints are the same as in Table 5.1, with a 60-epoch training budget for the LCNet variants and 30 epochs for the other configurations.

All seven configurations achieved IoUs above 92% for *Sky*, 90% for *Ridge*, and 86% for *Sand/Soil*, including the compact LCNet models. SegFormer-B0 stood out on *Hole* and *Track*, reaching 78.11% and 75.96%, whereas DeepLabV3+ with MobileNetV2 produced the highest *Rover* IoU, 84.99%. The main remaining weakness was *Rock*,

for which IoU stayed below 28% across all configurations.

Figure 5.1 compares the seven models on an S5Mars test image, showing the original scene, its ground truth, and the predicted segmentation masks. The scene contains exposed bedrock, sandy soil, and scattered rocks, with similar colours and textures making the distinction between Rock and Bedrock particularly challenging. Most models reproduce the large division between bedrock and soil, while differences become thinner around individual rocks and local boundaries. The contrast is visible in the lower right area, where SegFormer-B0 and DeepLabV3 ResNet34 are the only models to recognise the isolated rock as Rock, with the latter recovering only part of it. The remaining models classify the object as Bedrock, while the LCNet variants also miss smaller rocks and divide the lower bedrock region into patches separated by soil.

## 5.3 Quantization Results

Table 5.3 compares the previous results with the PTQ and QAT deployment variants described in Section 2.7. Accuracy was still evaluated on the S5Mars test split, whereas latency, energy per image, and peak GPU memory were measured through separate inference benchmarks with a batch size of one and an input resolution of $512 \times 512$. LCNet was not included in this quantization campaign, given their already low parameter count and efficiency.

FP16 conversion left test accuracy essentially unchanged across all five models, with the largest absolute mIoU difference remaining below 0.04%. Across the five FP32-FP16 comparisons, the reported energy reduction ranged from 40.5% to 74.5%. PTQ introduced larger, model-dependent accuracy changes. U-Net with ResNet34 lost 0.04%, SegFormer-B0 gained 0.05%, and DeepLabV3 with ResNet34 lost 0.23%. The two MobileNetV2 configurations were more sensitive in these experiments: U-Net lost 3.02% and DeepLabV3+ lost 4.93%, despite their lower latency and energy consumption.

The selected QAT configurations exceeded the corresponding FP32 test mIoU for four of the five models. SegFormer-B0 reached 76.93%, an increase of 1.38%, while DeepLabV3+ with MobileNetV2 reached 65.63%, an increase of 1.57%. DeepLabV3 with ResNet34 remained slightly below its FP32 counterpart, with a decrease of 0.17 points. These results describe the complete QAT optimization procedure, which included supervised fine-tuning and teacher distillation.

Figure 5.2 shows test mIoU versus measured GPU energy per image for the 20 examined configurations. Among these recorded points, QAT DeepLabV3+ and QAT SegFormer-B0 define the observed accuracy-energy frontier: neither has an alternative with both no greater energy consumption and no lower mIoU. DeepLabV3+ required the least measured energy, 115.34 mJ per image, whereas SegFormer-B0 provided the highest test mIoU at 291.76 mJ per image. Their comparison illustrates the remaining choice between a lower inference energy budget and more accurate segmentation, rather than identifying one configuration as preferable for every deployment requirement.

Resource improvements were not uniform across all measurements. For U-Net with ResNet34, the QAT engine reduced latency and energy but used 1106 MiB of

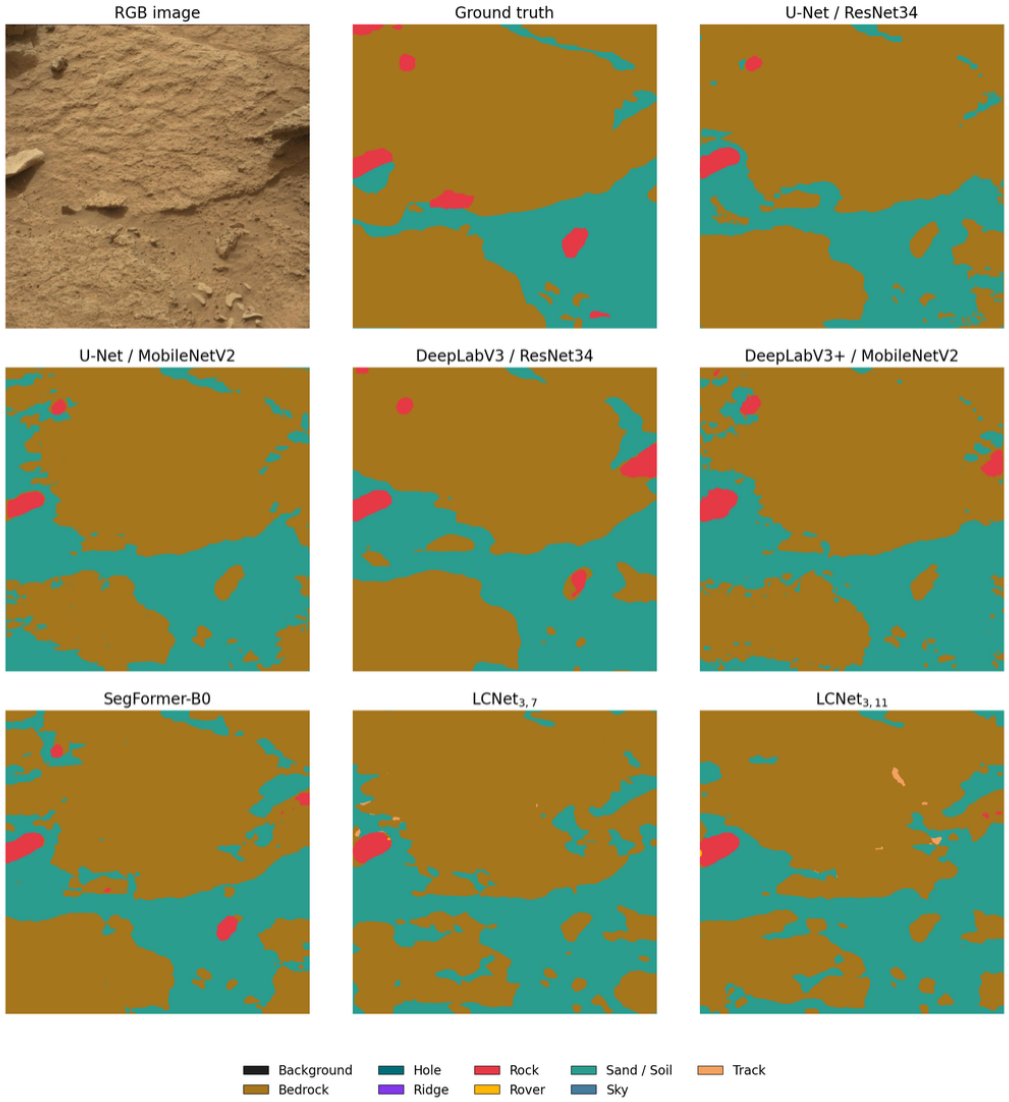



**Figure 5.1.** Qualitative comparison on an S5Mars test image, showing the RGB input, ground truth, and predictions of the seven evaluated configurations with a shared class colour mapping.

peak GPU memory, compared with 872 MiB for FP16. DeepLabV3 QAT also used more energy than its PTQ counterpart, 523.71 versus 455.34 mJ per image, despite its slightly higher accuracy. Since the execution backends and floating point fallback differ, these comparisons concern complete deployment configurations rather than isolating the effect of bit width. Furthermore, energy was estimated over a sustained GPU inference loop, includes device idle consumption, and excludes CPU energy. It is not an end-to-end system measurement or evidence of equivalent savings on rover flight hardware.

| Variant | Test mIoU (%) | ΔmIoU (pp) | Latency (ms) | Energy (mJ/image) | GPU peak (MiB) |
|---|---|---|---|---|---|
| *U-Net (ResNet34)* | | | | | |
| FP32 | 64.55 | – | 16.30 | 1213.08 | 1128 |
| FP16 | 64.55 | 0.00 | 9.19 | 659.76 | 872 |
| PTQ INT8 | 64.51 | -0.04 | 5.82 | 304.10 | 888 |
| QAT INT8 | 65.24 | +0.69 | 4.81 | 231.42 | 1106 |
| *U-Net (MobileNetV2)* | | | | | |
| FP32 | 64.57 | – | 11.87 | 878.80 | 1364 |
| FP16 | 64.61 | +0.04 | 8.16 | 523.07 | 1236 |
| PTQ INT8 | 61.55 | -3.02 | 4.17 | 199.96 | 1168 |
| QAT INT8 | 65.00 | +0.42 | 3.57 | 162.66 | 912 |
| *DeepLabV3 (ResNet34)* | | | | | |
| FP32 | 73.79 | – | 56.27 | 3977.25 | 414 |
| FP16 | 73.79 | 0.00 | 14.03 | 1016.14 | 284 |
| PTQ INT8 | 73.55 | -0.23 | 7.19 | 455.34 | 856 |
| QAT INT8 | 73.62 | -0.17 | 8.65 | 523.71 | 1130 |
| *DeepLabV3+ (MobileNetV2)* | | | | | |
| FP32 | 64.07 | – | 8.84 | 653.72 | 706 |
| FP16 | 64.07 | 0.00 | 6.03 | 377.88 | 706 |
| PTQ INT8 | 59.14 | -4.93 | 4.02 | 159.19 | 628 |
| QAT INT8 | 65.63 | +1.57 | 3.01 | 115.34 | 450 |
| *SegFormer-B0* | | | | | |
| FP32 | 75.55 | – | 11.88 | 835.72 | 1550 |
| FP16 | 75.55 | 0.00 | 7.17 | 432.07 | 1294 |
| PTQ INT8 | 75.60 | +0.05 | 5.92 | 327.99 | 1118 |
| QAT INT8 | 76.93 | +1.38 | 5.66 | 291.76 | 1008 |

**Table 5.3.** Accuracy and inference resource measurements for the selected configurations. Latency is the mean forward pass time with inputs already on the device, excluding preprocessing. Peak GPU memory includes the resident model or engine and buffers. FP32 and FP16 use ONNX Runtime CUDA, PTQ uses ONNX Runtime TensorRT with fallback, and QAT uses TensorRT directly. INT8 denotes selective mixed-precision execution, with Conv-only quantization for SegFormer-B0.

## 5.4 Ablation Studies

The ablation studies investigate how the training strategy and quantization scope affect segmentation accuracy, providing an experimental basis for the configurations adopted in this study. The comparisons examine alternative loss formulations and class-weighting schemes, encoder initialization, data augmentation, and oversampling to assess whether these choices improve mIoU relative to their default settings. Quantization experiments were also conducted to test whether extending INT8 conversion to additional model components preserves segmentation accuracy. Because of time constraints, some experiments were conducted on a subset of the segmentation models evaluated in this study.

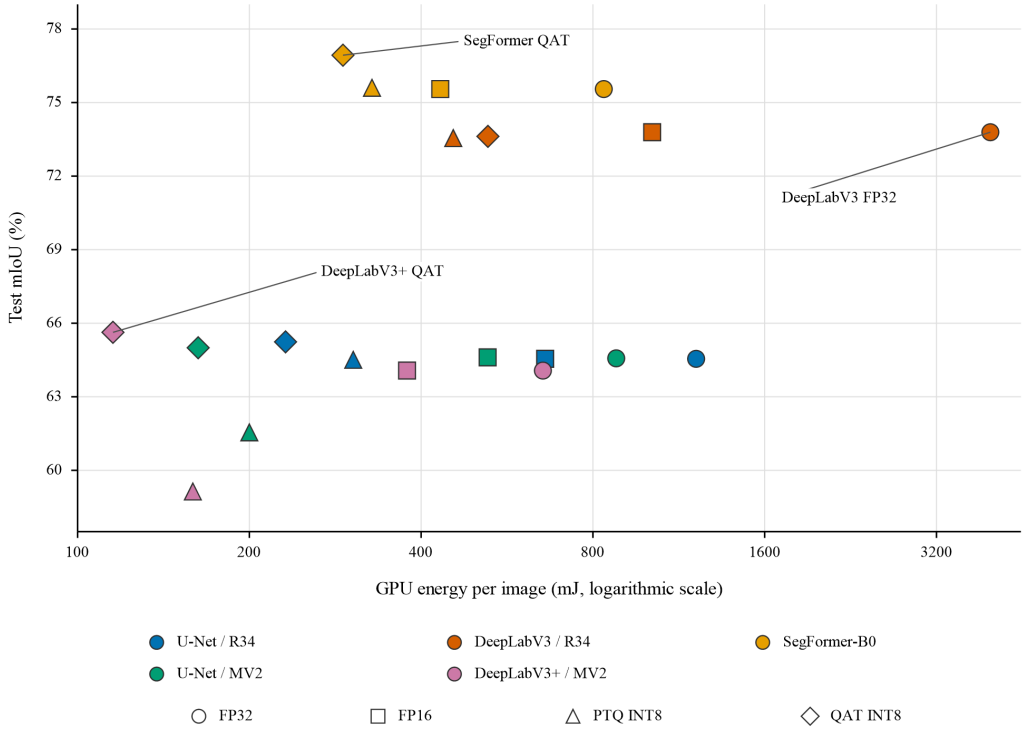



**Figure 5.2.** Test mIoU versus GPU energy per image for the various configurations, with energy shown on a logarithmic scale. Colours identify the model and marker shapes identify the precision or quantization strategy. Lower energy and higher mIoU are preferable. The two QAT annotations identify the best configurations among the reported measurements, while the DeepLabV3 FP32 annotation highlights its higher energy consumption.

### 5.4.1 Loss Function and Class Weighting

The initial loss comparison examined the combination of Cross-Entropy and Generalized Dice Loss (GDL) over Cross-Entropy alone for both SegFormer-B0 and U-Net with ResNet34, using the checkpoint with the highest validation mIoU for each run. As shown in Table 5.4, test mIoU increased from 69.2% to 73.7% for SegFormer-B0 and from 64.5% to 70.1% for U-Net with ResNet34. These trials motivated the use of the combined objective with $\alpha = 0.5$, which gives equal coefficients to pixel-wise classification and region-overlap supervision. Standalone GDL was also tried during preliminary experimentation but was not stable, so no numerical result is reported for that configuration.

| Model | CE | CE + GDL |
|---|---|---|
| SegFormer-B0 | 69.2 | 73.7 |
| U-Net / ResNet34 | 64.5 | 70.1 |

**Table 5.4.** Approximate test mIoU (%). CE + GDL used inverse-square Dice weights.

Class weighting was tested to assess whether increasing the Dice contribution of classes occupying fewer pixels improved segmentation accuracy. Table 5.5 compares the three GDL weighting schemes for U-Net with ResNet34. Inverse-square weighting (`square`) achieved a test mIoU of 70.1%, compared with 64.2% for uniform weighting and 57.2% for inverse-class-volume weighting (`simple`). These exploratory results motivated the use of inverse-square weighting in subsequent experiments. The interaction between this weighting scheme and oversampling is examined separately below.

| Model | Inverse square | Inverse volume | Uniform |
|---|---|---|---|
| U-Net / ResNet34 | 70.1 | 57.2 | 64.2 |

**Table 5.5.** Approximate test mIoU (%) for U-Net with ResNet34 using CE + GDL and the three Dice-weighting schemes.

### 5.4.2 Effect of Pretraining

The initialization experiment assessed whether ImageNet-pretrained encoder weights provided a useful starting point for learning Martian scene segmentation. U-Net with ResNet34 trained from random initialization reached a test mIoU of 51.6%, much below the pretrained U-Net results reported above. This observation is consistent with a benefit from pretrained initialization for the tested model and supports keeping it in subsequent experiments.

### 5.4.3 Effect of Data Augmentation

The augmentation experiment tested whether adding transformed versions of the training images could help SegFormer-B0 perform better on unseen S5Mars images. The tested policy combined horizontal and vertical flips with random right-angle rotations. With standard sampling and inverse-square Dice weights, enabling this policy reduced validation mIoU from 80.28% to 73.99%, as shown in Table 5.6. Test mIoU also fell from 75.55% to 66.86%. The extra geometric variation therefore did not improve accuracy in this experiment, therefore motivating keeping data augmentation disabled in the default configuration, while the test result shows the same pattern on unseen images.

The oversampling experiment tested whether showing images containing rare classes more often during training could improve SegFormer-B0 accuracy on unseen S5Mars images. It also examined whether uniform Dice weights worked better than inverse-square weights when oversampling was enabled. As shown in Table 5.6, oversampling with inverse-square weights produced lower validation and test mIoU than standard sampling: 70.64% versus 80.28% on validation and 71.02% versus 75.55% on test. Switching to uniform weights raised the oversampled run to 79.28% on validation and 72.90% on test, but both scores remained below the default. These results support keeping standard sampling with inverse-square Dice weights in the default configuration, with the validation and test scores showing the same trend.

| Sampling | Dice weights | Augmentation | Val. mIoU (%) | Test mIoU (%) |
|---|---|---|---|---|
| Standard | `square` | Disabled | 80.28 | 75.55 |
| Standard | `square` | Enabled | 73.99 | 66.86 |
| Oversampling | `square` | Disabled | 70.64 | 71.02 |
| Oversampling | `uniform` | Disabled | 79.28 | 72.90 |

**Table 5.6.** SegFormer-B0 training variants evaluated using the checkpoints with the highest validation mIoU in their respective training histories. All variants used the combined CE-GDL loss function. `square` denotes inverse-square class-volume weighting within GDL. Augmentation and oversampling were restricted to training, leaving validation and test samples unchanged.

The benefit of uniform weights was observed within the oversampled runs, probably due to the work of square weights being done in part by oversampling.

## 5.5 Cross-Mission Generalization on MarsSeg

The cross-mission experiment was designed to test whether a model trained on one rover mission could recognize the same terrain categories on another mission without additional training. This matters when labelled images from a new mission are scarce, and it also tests whether good results on the training mission translate into reliable generalization. MarsSeg MSL and MER were chosen because their Mars-Bench releases share the same terrain labels [3]. S5Mars contains only Curiosity imagery and uses a different taxonomy, which would require remapping labels for a comparable cross-mission test.

SegFormer-B0, U-Net with ResNet34, and DeepLabV3+ with MobileNetV2 were each trained separately on MSL and MER. For each run, the checkpoint selected on the source validation split was tested on both the source mission and the other mission, with the same preprocessing and output labels. The cross-mission test used no target domain fine-tuning, giving a zero-shot measure of transfer. Models trained on the target mission provided a reference on the same test images, as reported in Table 5.7.

| Model | MSL training | | MER training | |
|---|---|---|---|---|
| | MSL test | MER test | MER test | MSL test |
| SegFormer-B0 | 70.63 | **52.06** | 66.75 | **48.11** |
| U-Net / ResNet34 | **71.80** | 27.43 | **67.76** | 13.82 |
| DeepLabV3+ / MobileNetV2 | 68.98 | 38.27 | 65.10 | 12.86 |

**Table 5.7.** Test mIoU (%) for the MarsSeg cross-mission experiments. Each training mission identifies a separately trained model, evaluated on both test datasets. Evaluation on the other mission is zero-shot, without target-domain fine-tuning. Bold values indicate the highest recorded mIoU in each column.

In-domain results were similar across the three configurations, with U-Net ranking first on both missions. Once the mission changed, SegFormer-B0 performed substantially better than both convolutional models in both directions. The CNNs struggled especially when trained on MER and tested on MSL, while SegFormer showed a more balanced transfer across the two missions. This answers the main question of the experiment, showing that high in-domain accuracy alone did not ensure good generalization, as even SegFormer lost accuracy compared with its target-trained references.

The smaller MER training set may partly explain why transfer from MER to MSL was harder for the convolutional models. With fewer examples, models may learn features that are too specific to the source images. Acquisition also differs: MSL provides RGB Mastcam images, whereas MER provides grayscale Navcam/Pancam images, and the images differ in their original dimensions [3]. Such differences may make learned colour or texture patterns less useful on the other mission.

SegFormer's stronger transfer is consistent with its ability to combine local detail with larger scene context [12], together with its ADE20K segmentation pretraining. This initialization already provides features learned for pixel-level scene understanding, whereas the CNN encoders started from ImageNet classification weights, as described in Section 2.4. Both factors may help when mission-specific training data are limited or image appearance changes. The results therefore support the evaluated Transformer configuration for cross-mission generalization.

# Chapter 6

# Evaluation of Synthetic Damage Inspection

## 6.1 Experimental Setup

The experiments used the synthetic dataset described in Section 3.6, preserving its original training, validation, and test assignments when filtering by camera pose. The main comparison of PatchCore Small and Large, EfficientAD-S, SuperSimpleNet, and TinyGLASS focused on pose A, which represents a controlled acquisition setting while retaining variations in wheel identity, lighting, and surface wear. This subset contains 2,800 normal training images, 296 normal and 96 anomalous validation images, and 404 clean-anomalous pairs in the test split, for a total of 808 test images.

Table 6.1 defines the reference configurations and their abbreviations, which are reused throughout the chapter. The pose-A inputs use the fixed $720 \times 720$ crop described in Section 4.2, followed by model-specific preprocessing. The models therefore share the same image population, although their input resolutions and spatial coverage differ.

| ID | Model | Poses | Input (px) | Setting |
|---|---|---|---|---|
| PC-S | PatchCore Small | A | $384 \times 512$ | 50,000 Memory Bank |
| PC-L | PatchCore Large | A | $224 \times 224$ | 50,000 Memory Bank |
| EAD-S | EfficientAD-S | A | $256 \times 256$ | Global calibration |
| SSN | SuperSimpleNet | A | $256 \times 256$ | Top-1% score |
| TG-384 | TinyGLASS | A | $384 \times 384$ | $48 \times 48$, $\sigma = 1$ |
| TG-512 | TinyGLASS | A | $512 \times 512$ | $32 \times 32$, $\sigma = 4$ |

**Table 6.1.** Reference configurations for synthetic damage inspection. TinyGLASS settings indicate the feature-grid size and Gaussian smoothing parameter.

Both PatchCore configurations used a candidate reservoir capped at 1,000,000 descriptors, from which a memory bank of 50,000 descriptors was retained, with Gaussian smoothing set to $\sigma = 4$. After the fixed pose-A crop, PC-S resized the image directly to $384 \times 512$, whereas PC-L resized the shorter side to 256 pixels before applying a $224 \times 224$ centre crop. Image-score reweighting used nine

neighbours for PC-S and was disabled for PC-L. The SSN reference used a Perlin threshold of 0.2 without restricting synthetic perturbations to the target wheel mask. Both TinyGLASS references used wheel restricted, hole like LAS perturbations, sampling small, medium, and large severities with probabilities of 70%, 25%, and 5%, respectively. These probabilities apply to training perturbations, not to the composition of the rendered evaluation splits. TG-384 used layer2 features, whereas TG-512 used layer3 features, for efficiency purposes, with $\sigma = 4$ selected through a post hoc validation comparison of smoothing values.

SuperSimpleNet was trained with a batch size of 16 for up to 100 epochs, while TinyGLASS used batches of eight for up to 640 epochs. For these models, the checkpoint with the highest validation Image AUROC was kept and subsequently evaluated on the corresponding test split. Early stopping used a patience of five validation checks for SuperSimpleNet and 40 for TinyGLASS. EfficientAD-S instead used the final checkpoint of a fixed 70,000-step run, followed by calibration on the 296 normal validation images. PatchCore constructed its memory bank once for each configuration and therefore did not require an epoch-wise checkpoint search. The test results reported in the following sections use these chosen model checkpoints and the metrics defined in Section 4.4.

Efficiency benchmarks used an NVIDIA T4 GPU in Kaggle, processing one image at a time. Latency was averaged over 50 synchronized measurements after 10 warm-up runs, both for prediction on an input already in GPU memory and for the complete evaluation pipeline, including image and mask loading, preprocessing, and transfer to the GPU. Energy was measured separately during repeated prediction, with a 5-second warm-up followed by three 30-second measurement windows. NVML provided the GPU energy counter, with energy per image calculated by dividing the total GPU energy by the number of predictions across the three windows. This value covered the whole GPU, including idle consumption, and excluded CPU processing and input transfer.

## 6.2 Model Comparison

Table 6.2 compares the test results of the configurations defined in Table 6.1, separating image damage detection from pixel localization.

| Configuration | Image AUROC | Image AP | Pixel AUROC | Pixel AP | AUPRO |
|---|---|---|---|---|---|
| PC-S | 79.28 | 77.49 | **99.78** | 41.05 | **99.10** |
| PC-L | **86.43** | **88.63** | 99.43 | **42.89** | 95.44 |
| EAD-S | 66.50 | 64.74 | 96.36 | 9.00 | 84.98 |
| SSN | 82.72 | 86.08 | 96.97 | 19.10 | 87.53 |
| TG-384 | 77.52 | 79.28 | 97.21 | 25.48 | 90.24 |
| TG-512 | 77.33 | 81.12 | 97.92 | 27.59 | 92.01 |

**Table 6.2.** Damage detection and localization percentages on the pose-A test split. AUPRO is normalized up to FPR 0.30.

PC-L achieved the highest Image AUROC and Image AP, 86.43% and 88.63%,

followed by SSN at 82.72% and 86.08%. PC-S obtained a lower Image AUROC of 79.28%, while the two TinyGLASS configurations produced similar detection results, with Image AUROC values of 77.52% for TG-384 and 77.33% for TG-512. EAD-S remained below the other configurations on both image detection metrics.

The localization results distinguish the two PatchCore configurations more clearly. PC-L obtained the highest Pixel AP, 42.89%, whereas PC-S reached the highest Pixel AUROC, 99.78%, and AUPRO, 99.10%, with a Pixel AP of 41.05%. PC-L therefore provided the strongest image discrimination, while PC-S achieved better regional coverage. Within TinyGLASS, TG-512 increased Pixel AP from 25.48% to 27.59% and AUPRO from 90.24% to 92.01%, maintaining nearly the same Image AUROC as TG-384. Both TinyGLASS configurations localized damage more effectively than SSN according to Pixel AP and AUPRO, even though SSN performed better at anomalous image detection. EAD-S obtained a Pixel AUROC of 96.36% but only 9.00% Pixel AP, showing that a high Pixel AUROC can coexist with limited localization precision.

Alongside detection and localization performance, Table 6.3 reports prediction latency, peak allocated GPU memory, and energy per image for the various configurations.

| Configuration | Prediction latency (ms) | Peak allocated GPU memory (MiB) | Energy (J/image) |
|---|---|---|---|
| PC-S | 101.21 | 277.53 | 6.849 |
| PC-L | 62.89 | 385.25 | 4.116 |
| EAD-S | 57.72 | 878.45 | 4.032 |
| SSN | 27.54 | 219.69 | 1.939 |
| TG-384 | **5.13** | **38.72** | **0.239** |
| TG-512 | 6.29 | 55.03 | 0.369 |

**Table 6.3.** Prediction latency, peak allocated GPU memory, and GPU energy per image, with preprocessed inputs already on the GPU. Memory includes the model and other live tensors, as reported by the PyTorch allocator.

TG-384 had the lowest measured cost across all three resources, requiring 5.13 ms per image, 38.72 MiB of peak GPU memory, and 0.239 J per image. TG-512 required 6.29 ms, 55.03 MiB, and 0.369 J per image, with higher Pixel AP and AUPRO and nearly the same Image AUROC. The two TinyGLASS configurations therefore offer a choice between the lower resource requirements of TG-384 and the improved localization of TG-512. Both achieved better Pixel AP and AUPRO than SSN and EAD-S at a lower inference cost. This efficiency is consistent with TinyGLASS's compact architecture for edge deployment [41] and our use of fixed feature pooling without a separate trainable adaptor.

SSN offered an intermediate compromise for image-level detection: it reached 82.72% Image AUROC at 27.54 ms per image, compared with 86.43% at 62.89 ms for PC-L. These results favour TG-384 when inference resources are the main constraint, TG-512 when its localization improvement justifies the additional cost, and PC-L when detection quality has priority.

PC-S reduced peak GPU memory relative to PC-L, from 385.25 to 277.53 MiB, but required more time and energy per image. Its 101.21 ms latency and 6.849 J energy consumption were the highest measured values, so its advantage in regional coverage came with a greater inference cost. This behaviour is consistent with its denser feature grid, which produces more local descriptors to compare against the memory bank. EAD-S required the largest memory peak, without a corresponding advantage in detection or localization in this comparison.

To examine how performance varies across dataset conditions, Table 6.4 groups the test results by hole severity, lighting preset, and normal surface wear. Image AUROC describes detection within each group, while Pixel AP examines the localization of the corresponding damage regions.

| Group | Pairs | Metric | PC-S | PC-L | EAD-S | SSN | TG-384 | TG-512 |
|---|---|---|---|---|---|---|---|---|
| *Hole severity* | | | | | | | | |
| Small | 208 | Image AUROC | 78.59 | 80.20 | 62.90 | 77.15 | 74.15 | 74.09 |
| | | Pixel AP | 33.09 | 19.76 | 3.22 | 11.28 | 16.00 | 21.13 |
| Medium | 157 | Image AUROC | 78.99 | 92.07 | 70.10 | 87.97 | 80.10 | 79.11 |
| | | Pixel AP | 44.38 | 40.22 | 14.50 | 21.14 | 30.47 | 32.98 |
| Large | 39 | Image AUROC | 85.21 | 99.80 | 71.99 | 91.91 | 84.52 | 87.18 |
| | | Pixel AP | 55.27 | 68.93 | 14.46 | 27.04 | 34.01 | 35.13 |
| *Lighting preset* | | | | | | | | |
| Clear | 97 | Image AUROC | 82.30 | 84.71 | 67.70 | 83.61 | 79.66 | 81.29 |
| | | Pixel AP | 42.23 | 44.78 | 10.06 | 22.69 | 26.66 | 33.19 |
| Dusty | 307 | Image AUROC | 80.04 | 87.37 | 67.49 | 82.94 | 78.51 | 77.24 |
| | | Pixel AP | 43.60 | 43.30 | 11.47 | 18.38 | 28.25 | 26.66 |
| *Normal surface wear* | | | | | | | | |
| Current | 85 | Image AUROC | 83.64 | 91.29 | 65.51 | 80.86 | 75.82 | 79.33 |
| | | Pixel AP | 42.34 | 49.25 | 9.11 | 19.36 | 18.33 | 28.78 |
| Light wear | 179 | Image AUROC | 77.47 | 84.71 | 66.50 | 82.50 | 77.95 | 77.42 |
| | | Pixel AP | 39.92 | 39.48 | 8.70 | 19.76 | 28.24 | 26.46 |
| Evident wear | 140 | Image AUROC | 79.22 | 85.48 | 67.21 | 84.33 | 77.88 | 76.69 |
| | | Pixel AP | 43.19 | 45.48 | 11.25 | 18.41 | 27.38 | 29.24 |

**Table 6.4.** Test subgroup results (%) by hole severity, lighting, and surface wear, with clean-anomalous pair counts. Current denotes the surface without additional wear.

Hole severity produced the clearest common trend: all six configurations achieved higher Image AUROC on large holes than on small holes. PC-L showed the strongest separation, increasing from 80.20% for small holes to 92.07% and 99.80% for medium and large holes. Larger perforations occupy more of the visible wheel surface and are likely to retain a clearer anomaly signal after resizing and feature extraction. Small holes remain more difficult to distinguish from normal local texture, although PC-S preserved the highest Pixel AP on this group, 33.09%, followed by TG-512 at 21.13%. Both exceeded PC-L's 19.76%, so PC-L's higher aggregate Pixel AP is largely due to better localization of larger anomalous regions. TG-512 also improved Pixel AP over TG-384 at every severity, with the largest gain on small holes, from

16.00% to 21.13%. Pixel AP was higher for medium and large holes than for small holes across all six configurations, with part of this difference reflecting the greater proportion of anomalous pixels in larger defects.

Lighting had a less uniform effect. SSN obtained a Pixel AP of 22.69% under the clear preset and 18.38% under the dusty preset. TG-512's Image AUROC decreased from 81.29% to 77.24%, while its Pixel AP decreased from 33.19% to 26.66%. These differences are compatible with dust and illumination changes reducing the visual distinction between a perforation and the surrounding surface. Other configurations kept similar scores or improved on the dusty subgroup, therefore not representing a uniformly harder condition across the evaluated models.

Normal wear also affected the models differently. PC-L achieved its highest Image AUROC without additional wear, 91.29%, compared with 84.71% under light wear and 85.48% under evident wear. PC-S followed the same detection trend, which is consistent with scratches introducing local appearance changes that can resemble damage in feature matching. SSN instead had Image AUROC values between 80.86% and 84.33% across the three surface conditions, while TG-384 achieved higher Pixel AP under light and evident wear than without additional wear. TG-512's Pixel AP ranged from 26.46% under light wear to 29.24% under evident wear, with 28.78% without additional wear. The synthesis-based models may benefit from learning to distinguish local perturbations from the varied normal textures represented during training, although the observed wear results do not follow a common monotonic pattern.

For a visual comparison of the model predictions, Figure 6.1 reserves a common layout for a test image, its ground-truth damage mask, and the anomaly maps produced by the six reference configurations.

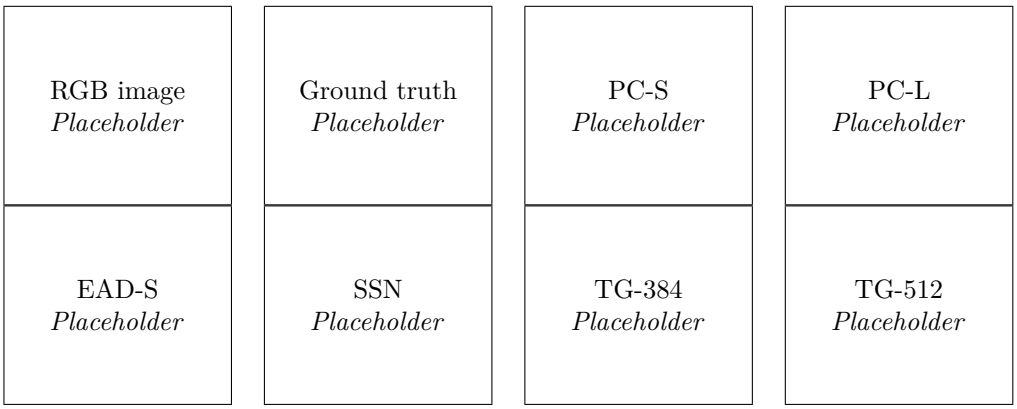



**Figure 6.1.** Placeholder grid for a qualitative comparison on a shared test sample, showing the RGB input, ground-truth damage mask, and anomaly maps from all six reference configurations.

## 6.3 Ablation Studies

The following experiments examined which changes improved damage detection and localization within each model family. The comparisons explain the choices used in the chosen configurations, considering the representation of normal appearance, the training perturbations, and the spatial detail available for small perforations.

### 6.3.1 Memory Bank and Smoothing in PatchCore

PatchCore was the first anomaly detector analysed, and these experiments were conducted using all camera poses, before the study adopted pose A as the common experimental setting. The memory bank size and smoothing studied in this stage were subsequently used for pose A experiments. The pose A comparison therefore used only the best configuration found.

The memory-bank comparison tested whether storing more examples of normal wheel appearance would improve detection and localization enough to justify the additional memory required to store the bank. The candidate reservoir was fixed at 1,000,000 descriptors, while the bank size varied as shown in Table 6.5. For the Small configuration, increasing the bank from 20,000 to 50,000 descriptors improved both validation Image AUROC and Pixel AP. A further increase to 100,000 doubled the storage without improving either validation metric. The Large preset showed a similar trade-off: 100,000 descriptors slightly improved validation Image AUROC, but reduced Pixel AP and required twice the bank memory. These results supported keeping 50,000 descriptors for both configurations, with the test results showing little benefit from the larger bank.

| Preset | Bank | Validation (%) | | Test (%) | | Bank (MiB) |
|---|---|---|---|---|---|---|
| | | Image AUROC | Pixel AP | Image AUROC | Pixel AP | |
| Small | 20,000 | 78.51 | 21.59 | 70.91 | 26.96 | 29.30 |
| Small | 50,000 | 81.89 | 22.68 | 72.81 | 27.56 | 73.24 |
| Small | 100,000 | 81.36 | 22.60 | 72.94 | 27.89 | 146.48 |
| Large | 50,000 | 89.73 | 31.39 | 77.94 | 30.95 | 195.31 |
| Large | 100,000 | 89.94 | 29.48 | 77.58 | 30.43 | 390.63 |

**Table 6.5.** Multi-pose PatchCore bank comparison with a reservoir of 1,000,000 descriptors and $\sigma = 4$. Small uses a $512 \times 512$ input and Large a $224 \times 224$ input. Memory refers to the bank alone.

Gaussian smoothing was examined separately to test whether reducing isolated high anomaly scores in the map would improve localization while preserving image detection. This comparison used the Small configuration with a bank of 100,000 descriptors. Table 6.6 shows that $\sigma = 4$ produced the highest Pixel AP on both splits, while test Image AUROC remained identical across the three settings. The result supports smoothing as a way to improve the spatial distribution of anomaly scores, and $\sigma = 4$ was kept for the subsequent PatchCore runs.

| $\sigma$ | Val Pixel AP | Test Pixel AP | Test Image AUROC |
|---|---|---|---|
| 0 | 20.15 | 24.75 | 72.61 |
| 1 | 20.57 | 25.16 | 72.61 |
| 4 | 23.05 | 27.69 | 72.61 |

**Table 6.6.** Smoothing comparison (%) for the multi-pose PC-S run with 100,000 descriptor bank.

### 6.3.2 Calibration and Scoring in EfficientAD-S

The EfficientAD-S experiments tested whether accounting for the normal response at each image position could reduce false anomaly responses from the surrounding scene and normal wheel structure. Spatial calibration used normal images to normalize the anomaly scores separately at each image position. It was combined with a static wheel region, and the image score was varied to examine whether averaging a small set of high responses would be more stable than taking the maximum. These scoring candidates were evaluated on the same run using validation Image AUROC.

Table 6.7 shows that spatial scoring improved validation discrimination over the global maximum. Averaging the highest 0.05% of responses within the wheel region gave the highest Image AUROC, although its result was almost identical to the spatial maximum. Increasing the averaging fraction reduced Image AUROC slightly, suggesting that the strongest local responses carried most of the useful detection signal.

| | Global max | Spatial max | Top 0.05% | Top 0.1% | Top 0.25% | Top 0.5% |
|---|---|---|---|---|---|---|
| Val Image AUROC | 80.22 | 84.98 | 85.00 | 84.93 | 84.83 | 84.64 |

**Table 6.7.** EfficientAD-S validation scoring comparison percentages for the spatial-calibration experiment. Top fractions denote averaging within the static wheel region.

The complete spatial and global training strategies were also compared to determine whether the improvement in validation detection was accompanied by better damage localization. The spatial strategy reserved normal training images for calibration, while the global strategy used all normal training images and calibrated after training on normal validation images. Both runs used pose A, the $720 \times 720$ crop, and a fixed 70,000-step budget.

As shown in Table 6.8, the spatial strategy achieved higher validation Image AUROC, but the global strategy produced much higher validation Pixel AP. The test results showed the same localization advantage and also favoured the global strategy for image detection. These results support the chosen global configuration as a compromise between detection and localization. One possible explanation is that position specific normalization is more sensitive to changes in lighting and surface appearance, whereas global calibration applies a common scale across image

positions.

| Strategy | Validation (%) | | Test (%) | |
|---|---|---|---|---|
| | Image AUROC | Pixel AP | Image AUROC | Pixel AP |
| Spatial, top 0.05% | 85.00 | 5.44 | 61.37 | 2.90 |
| Global, max | 81.23 | 15.21 | 66.50 | 9.00 |

**Table 6.8.** Comparison of the complete EfficientAD-S calibration and training strategies on pose A.

### 6.3.3 Training and Scoring in SuperSimpleNet

The SuperSimpleNet experiments explored whether adopting a single canonical camera pose could simplify wheel inspection by reducing variations in normal appearance. Since the models learn from clean images, a more consistent view could make it easier to distinguish localized damage from changes due to viewpoint. Pose A was therefore tested alongside the multi-pose setting, and a fixed crop before resizing was introduced to examine whether giving the wheel more of the input image would preserve useful damage detail and reduce the influence of the background. Table 6.9 compares the four pose and crop combinations at $256 \times 256$. The pose specific comparisons describe different image populations, while the crop comparison within each population shows the effect of concentrating the input on the wheel.

Pose A with cropping achieved the highest Image AUROC and Pixel AP in this comparison. Cropping was particularly useful for localization on pose A, increasing test Pixel AP from 7.27% to 19.49%. The modest detection improvement from cropping the multi-pose inputs also indicates that removing background alone did not resolve the difficulty introduced by different views. Together, these results support the controlled pose A acquisition and cropped input used for the SSN configuration and subsequent experiments with the other models.

| Metric | All poses No crop | All poses Crop | Pose A No crop | Pose A Crop |
|---|---|---|---|---|
| Val Image AUROC | 56.63 | 59.15 | 77.64 | 84.07 |
| Val Pixel AP | 8.77 | 11.62 | 9.18 | 18.44 |
| Test Image AUROC | 54.05 | 54.95 | 65.26 | 69.74 |
| Test Pixel AP | 8.33 | 11.40 | 7.27 | 19.49 |

**Table 6.9.** SuperSimpleNet acquisition comparison (%) at $256 \times 256$, before changing the image score formulation.

The number of training epochs was extended to test whether the initial configuration needed more optimization, and the input was increased to $384 \times 384$ to test whether additional spatial detail would improve localization. Table 6.10 shows that neither change improved validation image discrimination. At 200 epochs, the larger input increased test AUPRO but reduced Pixel AP, so its larger regional coverage

came with less precise pixel localization. The 100 epoch setting with $256 \times 256$ input size was chosen as the basis for the subsequent scoring and perturbation experiments.

| Input | Epochs | Val Image AUROC | Val Pixel AP | Test Pixel AP | Test AUPRO |
|---|---|---|---|---|---|
| $256 \times 256$ | 100 | 84.07 | 18.44 | 19.49 | 88.98 |
| $256 \times 256$ | 200 | 74.55 | 20.81 | 20.62 | 87.19 |
| $384 \times 384$ | 200 | 70.71 | 17.17 | 16.88 | 91.59 |

**Table 6.10.** SuperSimpleNet number of epochs and input resolution comparisons on cropped pose A images.

The image score was then reconsidered because a small localized hole may produce a useful anomaly map without a strong response from the learned classification head. The same checkpoint was therefore evaluated using the mean of the highest 1% of responses on the native map. Table 6.11 shows an improvement in both image metrics on validation and test, while the maps and localization metrics remained unchanged. This experiment supported using the map-based score in the subsequent SSN training runs.

| Image score | Validation | | Test | |
|---|---|---|---|---|
| | AUROC | AP | AUROC | AP |
| Classification head | 84.07 | 66.67 | 69.74 | 72.45 |
| Top 1% mean | 92.07 | 84.45 | 81.11 | 84.72 |

**Table 6.11.** Image-score comparison on the same SuperSimpleNet run, without retraining.

The synthetic perturbations were also varied to test how their extent and location affected learning. Restricting Perlin perturbations to the target wheel was intended to focus training on the structure being inspected. Within this setting, lowering the Perlin threshold from 0.6 to 0.2 tested whether a larger perturbed region would provide a stronger training signal. The high-threshold run produced weak detection and localization, whereas the lower threshold substantially improved both (Table 6.12). A plausible explanation is that the combination of a high threshold and wheel restriction left too little useful anomalous content on the feature grid.

Removing the wheel restriction at threshold 0.2 tested whether perturbations over a wider range of normal features would improve pixel discrimination. Validation Wheel Pixel AP increased from 15.45% to 18.49%, and test Pixel AP increased from 16.78% to 19.10%. Image detection decreased slightly, as did test AUPRO, indicating a compromise between precision and regional coverage. The unrestricted variant was chosen for its stronger localization precision.

Finally, smoothing was checked to determine whether it could improve spatial consistency without affecting the selected image score. The validation sweep over $\sigma = 0, 1, 2, 4$ favoured $\sigma = 4$ for both restricted and unrestricted perturbations, using Wheel Pixel AP for selection. Image metrics remained unchanged because the

| Perlin threshold | Wheel restriction | Val Image AUROC | Val Wheel Pixel AP | Test Image AUROC | Test Pixel AP |
|---|---|---|---|---|---|
| 0.6 | Yes | 54.69 | 0.20 | 55.73 | 0.29 |
| 0.2 | Yes | 94.07 | 15.45 | 83.13 | 16.78 |
| 0.2 | No | 93.35 | 18.49 | 82.72 | 19.10 |

**Table 6.12.** SuperSimpleNet perturbation comparisons (%) on cropped pose A inputs, using the top-1% image score and $\sigma = 4$. Wheel restriction applies to training perturbations.

score was computed before smoothing. This supported using $\sigma = 4$ in the final SSN configuration.

### 6.3.4 Task Adaptation and Spatial Resolution in TinyGLASS

The first TinyGLASS experiment tested whether the constraints used to create artificial anomalies helped the model detect wheel damage. With the hypersphere constraint enabled, altered features were kept within a set distance range from a single centre representing normal appearance. This constraint was removed to test whether using that common centre limited the training examples that the model could learn from. GAS was still used, with local limits on the distance between each perturbed feature and its original normal feature. With the $900 \times 900$ crop, removing the hypersphere constraint improved Pixel AP on both validation and test, as shown in Table 6.13, supporting its removal from the subsequent configurations.

At the selected $720 \times 720$ crop, LAS texture perturbations were compared with irregular, darkened hole-like regions restricted to the target wheel. This experiment tested whether training anomalies closer to the appearance of the target defect would improve detection and localization. The hole variant improved both Image AUROC and Pixel AP on validation and test, supporting the use of these task specific perturbations.

| Crop (px) | Training setting | Validation (%) | | Test (%) | |
|---|---|---|---|---|---|
| | | Image AUROC | Pixel AP | Image AUROC | Pixel AP |
| 900 | Texture, hypersphere | 68.31 | 1.62 | 56.33 | 1.41 |
| 900 | Texture, local | 75.40 | 6.39 | 63.81 | 7.32 |
| 720 | Texture, local | 76.77 | 8.38 | 66.78 | 6.97 |
| 720 | Hole, initial | 80.60 | 9.01 | 69.72 | 11.46 |
| 720 | Hole, 50/40/10% | 83.35 | 14.40 | 72.04 | 14.71 |
| 720 | Hole, 70/25/5% | 83.74 | 15.18 | 72.55 | 14.67 |

**Table 6.13.** TinyGLASS training variants on pose A, using a $256 \times 256$ input, the $32 \times 32$ grid, and $\sigma = 4$. Local denotes the setting without the hypersphere constraint. The hole percentages refer to small, medium, and large perturbations.

The comparison of the size of perturbations varied the LAS probabilities for

small, medium, and large perturbations from 50%, 40%, and 10% to 70%, 25%, and 5%. The aim was to test whether showing the model more small synthetic defects during training would improve their localization. Validation Image AUROC and Pixel AP improved modestly, but test Pixel AP remained almost unchanged, and small hole Pixel AP decreased from 7.21% to 6.84%. The revised distribution was kept for its validation performance, but the result showed that perturbation frequency alone did not resolve the difficulty of small defects.

Image score aggregation was tested to check whether averaging high patch responses or using a quantile could suppress the effects of an outlier without losing sensitivity to small holes. On the LAS hole checkpoint, the maximum achieved validation Image AUROC of 80.60% and Image AP of 59.51%. Averaging the three highest responses gave an almost identical Image AUROC of 80.55%, but lower Image AP of 58.91%, while wider averages and the tested quantiles performed worse, motivating the use of the maximum.

The resolution experiments also examined the trade-off between accuracy and computational cost, testing whether the additional detail from larger inputs justified the extra processing required. Larger images could preserve small perforations more clearly, but increased the work done by the backbone. Both layer2 and layer3 features were used throughout, with either interpolation to the denser layer2 grid or pooling to the coarser layer3 grid. This comparison tested whether fewer patches could reduce the work done by the discriminator while preserving useful damage information. Table 6.14 reports the resulting detection and localization metrics.

| Input | Final Resolution | $\sigma$ | Val Image AUROC | Val Pixel AP | Test Image AUROC | Test Pixel AP |
|---|---|---|---|---|---|---|
| $256 \times 256$ | layer2, $32 \times 32$ | 1 | 83.74 | 16.53 | 72.55 | 15.72 |
| $384 \times 384$ | layer2, $48 \times 48$ | 1 | 88.36 | 26.05 | 77.52 | 25.48 |
| $384 \times 384$ | layer3, $24 \times 24$ | 1 | 87.58 | 25.98 | 76.91 | 24.18 |
| $384 \times 384$ | layer3, $24 \times 24$ | 4 | 87.58 | 26.48 | 76.91 | 24.57 |
| $512 \times 512$ | layer2, $64 \times 64$ | 1 | 83.16 | 24.08 | 73.81 | 23.55 |
| $512 \times 512$ | layer2, $64 \times 64$ | 4 | 83.16 | 24.56 | 73.81 | 24.07 |
| $512 \times 512$ | layer3, $32 \times 32$ | 1 | 85.57 | 28.26 | 77.33 | 27.01 |
| $512 \times 512$ | layer3, $32 \times 32$ | 4 | 85.57 | 28.84 | 77.33 | 27.59 |

**Table 6.14.** TinyGLASS input, grid, and smoothing comparisons on pose A with the $720 \times 720$ crop and LAS hole probabilities of 70%, 25%, and 5%. Image AUROC is shared by smoothing variants of the same checkpoint.

With $\sigma = 1$ and layer2 alignment, increasing the input from $256 \times 256$ to $384 \times 384$ improved detection and localization, with a relative increase of 62.1% in test Pixel AP. A further increase to $512 \times 512$ reduced test Pixel AP by 7.6%. Pooling to the layer3 grid reduced the number of patches to one quarter, with test Pixel AP decreasing by 5.1% at $384 \times 384$ but increasing by 14.7% at $512 \times 512$. The higher resolution was therefore useful when combined with pooling, which may reduce the influence of normal texture while retaining damage information. The reduced patch count lowers discriminator work and feature storage, while the backbone continues to process the full input.

Smoothing was compared on the same checkpoints to assess whether localization could improve without further training. The validation sweep at $256 \times 256$ with layer2 alignment favoured $\sigma = 1$ for regional coverage, so this setting was carried forward to TG-384. Increasing smoothing to $\sigma = 4$ produced small gains in validation and test Pixel AP for the $384 \times 384$ layer3 grid and both $512 \times 512$ grids. The $512 \times 512$ configuration with layer3 alignment achieved the highest validation and test Pixel AP among the settings shown, with a further relative gain of about 2% on test after stronger smoothing. Image detection remained unchanged because its score was computed on the native patch grid before smoothing. Together with the resource measurements in Table 6.3, these results support retaining TG-384 for its lower computational cost and TG-512 for its stronger localization.

# Chapter 7

# Discussion and Limitations

## 7.1 Main Findings

## 7.2 Sim-to-Real Gap

# Chapter 8

# Conclusions and Future Work

# Bibliography

[1] NASA Jet Propulsion Laboratory, "How to Drive a Mars Rover," *On a Mission*, Season 4, Episode 8, Aug. 25, 2022. [Online]. Available: https://www.jpl.nasa.gov/podcasts/on-a-mission-season-1/season-4-mars-rovers/episode-8-how-to-drive-a-mars-rover/.

[2] J. Zhang, L. Lin, Z. Fan, W. Wang, and J. Liu, "$S^5$Mars: Semi-Supervised Learning for Mars Semantic Segmentation," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 62, pp. 1–15, 2024, doi: 10.1109/TGRS.2024.3386756.

[3] M. Purohit, B. Gajera, V. Malaviya, I. Mehta, K. Kasodekar, J. Adler, S. Lu, U. Rebbapragada, and H. Kerner, "Mars-Bench: A Benchmark for Evaluating Foundation Models for Mars Science Tasks," in *Advances in Neural Information Processing Systems*, vol. 38, Datasets and Benchmarks Track, 2025, pp. 136254–136304, doi: 10.52202/085713-4098.

[4] Y. Xiong, X. Xiao, M. Yao, H. Cui, and Y. Fu, "Light4Mars: A Lightweight Transformer Model for Semantic Segmentation on Unstructured Environment Like Mars," *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 214, pp. 167–178, Aug. 2024, doi: 10.1016/j.isprsjprs.2024.06.008.

[5] R. M. Swan, D. Atha, H. A. Leopold, M. Gildner, S. Oij, C. Chiu, and M. Ono, "AI4MARS: A Dataset for Terrain-Aware Autonomous Driving on Mars," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 1982–1991, Jun. 2021, doi: 10.1109/CVPRW53098.2021.00226.

[6] J. Li, S. Zi, R. Song, Y. Li, Y. Hu, and Q. Du, "A Stepwise Domain Adaptive Segmentation Network With Covariate Shift Alleviation for Remote Sensing Imagery," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, pp. 1–15, 2022, Art. no. 5618515, doi: 10.1109/TGRS.2022.3152587.

[7] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention—MICCAI 2015*, Lecture Notes in Computer Science, vol. 9351, pp. 234–241, 2015, doi: 10.1007/978-3-319-24574-4_28.

[8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016, doi: 10.1109/CVPR.2016.90.

[9] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018, doi: 10.1109/CVPR.2018.00474.

[10] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," *arXiv preprint arXiv:1706.05587*, 2017.

[11] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder-decoder with atrous separable convolution for semantic image segmentation," in *Computer Vision—ECCV 2018*, Lecture Notes in Computer Science, vol. 11211, pp. 833–851, 2018, doi: 10.1007/978-3-030-01234-2_49.

[12] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "SegFormer: Simple and efficient design for semantic segmentation with transformers," in *Advances in Neural Information Processing Systems*, vol. 34, pp. 12077–12090, 2021.

[13] M. Shi, S. Lin, Q. Yi, J. Weng, A. Luo, and Y. Zhou, "Lightweight context-aware network using partial-channel transformation for real-time semantic segmentation," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 7, pp. 7401–7416, 2024, doi: 10.1109/TITS.2023.3348631.

[14] P. Iakubovskii, "Segmentation Models Pytorch," GitHub repository, 2019. [Online]. Available: https://github.com/qubvel-org/segmentation_models.pytorch

[15] NVIDIA, "SegFormer-B0 fine-tuned on ADE20K at 512 × 512," Hugging Face model repository. [Online]. Available: https://huggingface.co/nvidia/segformer-b0-finetuned-ade-512-512

[16] D. Haase and M. Amthor, "Rethinking Depthwise Separable Convolutions: How Intra-Kernel Correlations Lead to Improved MobileNets," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14588–14597, 2020.

[17] J. Li, K. Chen, G. Tian, L. Li, and Z. Shi, "MarsSeg: Mars surface semantic segmentation with multilevel extractor and connector," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 63, Art. no. 4501012, 2025, doi: 10.1109/TGRS.2025.3526630.

[18] S. M. Raza, S. M. H. Abidi, M. Masuduzzaman, and S. Y. Shin, "Lightweight Deep Learning for Visual Perception: A Survey of Models, Compression Strategies, and Edge Deployment Challenges," *Neurocomputing*, vol. 656, Art. no. 131357, 2025. doi: 10.1016/j.neucom.2025.131357.

[19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[20] C. H. Sudre, W. Li, T. Vercauteren, S. Ourselin, and M. J. Cardoso, "Generalised Dice overlap as a deep learning loss function for highly unbalanced segmentations," in *Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support*, Lecture Notes in Computer Science, vol. 10553, Springer, 2017, pp. 240–248, doi: 10.1007/978-3-319-67558-9_28.

[21] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. International Conference on Learning Representations (ICLR)*, 2019.

[22] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. International Conference on Learning Representations (ICLR)*, 2017.

[23] M. Yeung, E. Sala, C.-B. Schönlieb, and L. Rundo, "Unified Focal loss: Generalising Dice and cross entropy-based losses to handle class imbalanced medical image segmentation," *Computerized Medical Imaging and Graphics*, vol. 95, art. no. 102026, 2022, doi: 10.1016/j.compmedimag.2021.102026.

[24] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.

[25] Microsoft, "Quantize ONNX Models," *ONNX Runtime Documentation*. Available: https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html.

[26] NASA, "Routine Inspection of Rover Wheel Wear and Tear," *NASA Science*, April 27, 2016. Available: https://science.nasa.gov/resource/routine-inspection-of-rover-wheel-wear-and-tear/.

[27] A. Rankin, N. Patel, E. Graser, J.-K. F. Wang, and K. Rink, "Assessing Mars Curiosity Rover Wheel Damage," in *2022 IEEE Aerospace Conference (AERO)*, 2022, pp. 1–19. Available: https://robotics.jpl.nasa.gov/media/documents/fmwi-rankin-2022-0225-final.pdf.

[28] S. Lu and K. L. Wagstaff, "MSL Curiosity Rover Images with Science and Engineering Classes," version 2.1.0, Zenodo, 2020. doi: 10.5281/zenodo.4033453.

[29] NASA Visualization Technology Applications and Development, "Curiosity 3D Model," NASA Science, April 22, 2019. Available: https://science.nasa.gov/resource/curiosity-3d-model/.

[30] HiRISE, "Possible MSL Landing Site in Gale Crater," Digital terrain model and orthoimage products for observations `PSP_010573_1755` and `PSP_010639_1755`. Available: https://hirise.lpl.arizona.edu/dtm/dtm.php?ID=PSP_010573_1755.

[31] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23–30, 2017. doi: 10.1109/IROS.2017.8202133.

[32] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[33] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD – A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9584–9592. doi: 10.1109/CVPR.2019.00982.

[34] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards Total Recall in Industrial Anomaly Detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 14318–14328. Available: CVF Open Access.

[35] M. A. Mohammed Eltoum, E. Iqbal, Y. Zweiri, B. Moyo, and Y. Abdulrahman, "BladeSynth: A High-Quality Rendering-Based Synthetic Dataset for Aero Engine Blade Defect Inspection," *Scientific Data*, vol. 12, article 1268, 2025. doi: 10.1038/s41597-025-05563-y.

[36] Y. Zhou, J. Cui, B. Lian, H. Luo, Y. Zheng, and Z.-M. Lu, "Surface Anomaly Detection on Shock Absorber Connecting Rods Using an Enhanced PatchCore Algorithm," *IEICE Transactions on Information and Systems*, vol. E109-D, no. 5, pp. 707–711, 2026. doi: 10.1587/transinf.2025EDL8047.

[37] K. Batzner, L. Heckler, and R. König, "EfficientAD: Accurate Visual Anomaly Detection at Millisecond-Level Latencies," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2024, pp. 128–138. Available: CVF Open Access.

[38] S. Villanueva López, E. Soria-Olivas, and M. Sánchez-Montañés, "RGB Ensemble Strategies for Unsupervised Industrial Anomaly Detection on the AutoVI Dataset," *Electronics*, vol. 15, no. 10, article 2077, 2026. doi: 10.3390/electronics15102077.

[39] B. Rolih, M. Fučka, and D. Skočaj, "SuperSimpleNet: Unifying Unsupervised and Supervised Learning for Fast and Reliable Surface Defect Detection," in *Pattern Recognition (ICPR 2024)*, Lecture Notes in Computer Science, vol. 15310, Springer, 2025, pp. 47–65. doi: 10.1007/978-3-031-78192-6_4.

[40] B. Rolih, M. Fučka, and D. Skočaj, "No label left behind: a unified surface defect detection model for all supervision regimes," *Journal of Intelligent Manufacturing*, vol. 37, pp. 3189–3209, 2026. doi: 10.1007/s10845-025-02680-8.

[41] P. Bonazzi, R. Sutter, L. Capogrosso, M. Buob, and M. Magno, "TinyGLASS: Real-Time Self-Supervised In-Sensor Anomaly Detection," *arXiv preprint arXiv:2603.16451*, 2026. Accepted at the IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS), 2026. Available: arXiv:2603.16451v3.

[42] Q. Chen, H. Luo, C. Lv, and Z. Zhang, "A Unified Anomaly Synthesis Strategy with Gradient Ascent for Industrial Anomaly Detection and Localization," in *Computer Vision – ECCV 2024*, Lecture Notes in Computer Science, vol. 15125, Springer, 2025, pp. 37–54. doi: 10.1007/978-3-031-72855-6_3.

[43] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLoS ONE*, vol. 10, no. 3, article e0118432, 2015. doi: 10.1371/journal.pone.0118432.

[44] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, and C. Steger, "The MVTec Anomaly Detection Dataset: A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection," *International Journal of Computer Vision*, vol. 129, pp. 1038–1059, 2021. doi: 10.1007/s11263-020-01400-4.